

Multi-GPU Mixed-Precision Inexact Interior Point Methods for Large-Scale Dense Linear Programming

Author One¹

Author Two²

¹Affiliation One

²Affiliation Two

June 9, 2026

Abstract

We propose a multi-GPU mixed-precision inexact interior point method for large-scale dense linear programming. The normal-equations matrix is partitioned into diagonal blocks, each factorized on a separate GPU in low precision (FP32 or TF32) and used as a block-diagonal preconditioner for conjugate gradient or iterative refinement. The linear-solve tolerance is adaptively scheduled as a function of the IPM duality measure μ_k , from 10^{-4} in early iterations to 10^{-10} near optimality, with a guaranteed FP64 fallback. We prove polynomial convergence of the predictor-corrector IPM when the relative residual satisfies $\eta_k \leq \bar{c}\mu_k^\alpha$, and analyze how inexactness propagates through the primal and dual feasibility conditions. Experiments on dense instances up to $m = 200$, $n = 600$ show that fixed-accuracy low-precision and block-Jacobi solvers fail to converge, while the adaptive solver matches the exact baseline in both iteration count and solution quality. The framework draws on the error-management philosophy of quantum interior point methods, recasting it as a practical strategy for heterogeneous GPU computing.

1 Introduction

Linear programming (LP) is a fundamental technology in operations research, machine learning, quantitative finance, and engineering design. Modern applications routinely produce LP instances with tens of thousands of variables and constraints: portfolio optimization over large asset universes, sparse recovery and compressed sensing in signal processing, optimal transport in machine learning, and network flow problems in logistics and supply chain management, to name a few. When the constraint matrix is dense—as it is in many of these applications after preprocessing, basis transformations, or when the problem is naturally fully coupled—the computational bottleneck shifts decisively to the linear algebra at each iteration of the optimization algorithm.

Interior point methods (IPMs) [?, ?] are among the most reliable and theoretically well-understood algorithms for solving LPs. A primal-dual IPM follows the central path toward optimality by solving, at each iteration, a Newton system that, after elimination of the primal and dual slack variables, reduces to the $m \times m$ symmetric positive definite *normal equations*

$$\mathbf{M}_k \Delta \mathbf{y}_k = \mathbf{k}, \quad \mathbf{M}_k := \mathbf{A} \mathbf{D}_k^2 \mathbf{A}^T, \quad (1)$$

where $\mathbf{A} \in \mathbb{R}^{m \times n}$ is the constraint matrix, $\mathbf{D}_k = \text{diag}(\sqrt{x_1/s_1}, \dots, \sqrt{x_n/s_n})$ encodes the primal-dual scaling at iteration k , and $\mathbf{k}$ is the assembled right-hand side of the Mehrotra predictor-corrector scheme [?]. The matrix $\mathbf{M}_k$ must be “solved”—in the sense of computing $\Delta \mathbf{y}_k$ that satisfies (??)—at every IPM iteration, and this is by far the most expensive operation in the algorithm. In the codebase, $\mathbf{M}_k$ is formed by the function `compute_normal_equation` in `lp_problem.py`

as `A_scaled = A * D_diag[None, :]` followed by `A_scaled @ A_scaled.T`, exactly implementing $\mathbf{M}_k = (\mathbf{A}\mathbf{D}_k)(\mathbf{A}\mathbf{D}_k)^T$.

For large-scale *dense* LPs, where $m \approx 10^3\text{--}10^5$ and $\mathbf{A}$ has no exploitable sparsity pattern, the standard approach is to factor $\mathbf{M}_k$ via dense Cholesky decomposition $\mathbf{M}_k = \mathbf{L}^T\mathbf{L}$, which costs $\mathcal{O}(m^3)$ floating-point operations and must be repeated at every single IPM iteration (typically 15–40 iterations for a well-tuned implementation). The total cost of the Cholesky factorizations alone can easily reach $10^{12}\text{--}10^{15}$ operations, dominating the solution time. For instance, at $m = 10^4$, each Cholesky factorization involves approximately 1.7×10^{11} operations; across 20 IPM iterations, this amounts to 3.4×10^{12} operations, requiring minutes to hours on a single CPU core.

Two recent developments, one in hardware and one in theory, motivate a fundamentally different approach to this bottleneck.

Hardware trend: the GPU precision gap. Modern GPU architectures deliver extraordinarily high throughput for dense linear algebra operations (GEMM, SYRK, TRSM) through tensor cores [?]. However, the throughput is strongly precision-dependent: on the NVIDIA H100 GPU, FP64 (double precision) delivers approximately 34 TFLOPS, while FP32 delivers 67 TFLOPS and TF32 (tensor-float-32) reaches 990 TFLOPS—a nearly $30\times$ gap between the highest-throughput low-precision mode and the FP64 mode required for classical IPM linear algebra. This precision gap means that a Cholesky factorization performed entirely in FP64 on a GPU leaves most of the hardware’s computational capacity unused. If we could perform the bulk of the factorization in FP32 or TF32 and recover accuracy through a small number of correction steps in FP64, the effective throughput could increase by an order of magnitude or more.

Algorithmic trend: inexact IPM theory. Independently, the theory of *inexact* interior point methods [?, ?] has established that the Newton direction need not be computed to machine precision at every IPM iteration. It suffices that the relative residual of the linear solve,

$$\eta_k := \frac{\|\mathbf{M}_k \widehat{\Delta \mathbf{y}}_{k-k}\|_2}{\|\mathbf{r}_k\|_2}, \quad (2)$$

be bounded by a quantity that shrinks with the IPM’s progress toward optimality: $\eta_k \leq \bar{c} \mu_k^\alpha$, where $\mu_k = \mathbf{x}_k^T \mathbf{s}_k / n$ is the current duality measure. In plain terms, the linear solve can be quite inaccurate when the IPM is far from optimality (μ_k large), but must become progressively more accurate as the solver converges ($\mu_k \rightarrow 0$). This theoretical insight transforms the linear-solve accuracy from a *fixed* requirement (machine precision at every iteration) into a *dynamic* resource that can be allocated proportionally to the IPM’s needs. The residual η_k is computed in the code by `_compute_eta` in `linear_solver.py` as `np.linalg.norm(M @ delta_y - r) / np.linalg.norm(r)`, always evaluated in FP64 regardless of the internal precision used by the solver.

Our central observation is that these two developments are *complementary*. The GPU precision gap makes low-precision factorization fast; inexact IPM theory makes low-accuracy solves acceptable. What is needed is a bridge between them: a precision-and-accuracy scheduling mechanism that decides, at each IPM iteration, *how much* accuracy is required of the linear solve and *in what precision* to compute the factorization, given the current duality measure μ_k .

We propose such a bridge in this paper: the *multi-GPU mixed-precision inexact interior point method* for large-scale dense linear programming. The algorithm combines three ideas. First, the normal-equations matrix $\mathbf{M}_k$ is partitioned into p diagonal blocks, each assigned to a distinct GPU and factorized independently via dense Cholesky in low precision (FP32 or TF32), yielding a block-diagonal preconditioner $\mathbf{P}_k = \text{blkdiag}(\mathbf{M}_{11}, \dots, \mathbf{M}_{pp})$. This block-partition strategy

is implemented in `_build_block_diagonal_preconditioner` in `linear_solver.py`, which slices $\mathbf{M}_k$ into `num_blocks` contiguous diagonal blocks and factors each block via Cholesky. Second, the Newton step is computed by preconditioned conjugate gradient or mixed-precision iterative refinement, reusing the low-precision block factors to correct the solution—a *factorize-once, solve-many* pattern that amortizes the $\mathcal{O}(m^3)$ factorization cost across the predictor step, the corrector step, and all refinement corrections. Third, and most critically, the target accuracy for the linear solve is adaptively scheduled as a piecewise function of μ_k , implemented by `_mu_to_eta_target` in `linear_solver.py`: $\eta_{\text{target}}(\mu) = 10^{-4}$ when $\mu > 10^{-2}$ (early IPM, loose accuracy), $\eta_{\text{target}}(\mu) = 10^{-7}$ when $10^{-6} < \mu \leq 10^{-2}$ (mid IPM), and $\eta_{\text{target}}(\mu) = 10^{-10}$ when $\mu \leq 10^{-6}$ (late IPM, near-exact solves). A fallback to FP64 Cholesky guarantees convergence when the low-precision factorization cannot meet the target.

The approach is inspired, in part, by recent work on quantum interior point methods (QIPM) [?]. In QIPM, a quantum linear system algorithm (QLSA) provides an approximate Newton direction whose error is bounded in ℓ_2 -norm, and the QIPM literature has developed a detailed theory of how to schedule QLSA accuracy across IPM iterations. Our setting is strictly classical—the error source is floating-point round-off and block-diagonal approximation rather than quantum sampling noise—but the mathematical structure is isomorphic: both frameworks must cope with an inexact linear solve at each iteration, and both rely on the outer IPM’s ability to absorb bounded linear-system error. The adaptive tolerance schedule developed here can be viewed as a classical analogue of the QLSA precision schedule in QIPM, and our analysis of feasibility drift (Section ??) parallels the feasibility-preserving Newton reformulations studied in the QIPM literature.

The paper makes the following contributions. It presents a convergence analysis of the Mehrotra predictor-corrector IPM with inexact Newton directions governed by the condition $\eta_k \leq \bar{c}\mu_k^\alpha$, showing that polynomial complexity $\mathcal{O}(\sqrt{n} \log(1/\varepsilon))$ is retained (Section ??, Theorem ??). It develops a multi-GPU block-diagonal Cholesky preconditioner that trades off-diagonal coupling for embarrassingly parallel factorization across GPUs, analyzes its approximation quality through the generalized eigenvalue bound (??), and presents the `LowPrecisionCholeskySolver` and `BlockJacobiPCGSolver` algorithms (Section ??, Algorithms ?? and ??). It designs and implements a μ -driven adaptive refinement solver that dynamically adjusts the linear-solve accuracy target across three phases, mathematically encoding this as the piecewise function (??) and embedding it in an iterative refinement loop with a guaranteed FP64 fallback (Section ??, Algorithm ??). And it provides numerical experiments on dense LP instances up to $m = 200$, $n = 600$ that reveal a sharp empirical finding: fixed-accuracy low-precision and block-Jacobi solvers *fail to converge on every problem size tested*, while the adaptive solver tracks the exact baseline in both iteration count and solution quality (Section ??, Table ??).

The remainder of the paper is organized as follows. Section ?? reviews the primal-dual IPM framework, the normal-equations reduction, the Mehrotra predictor-corrector algorithm, and the fraction-to-boundary step rule, establishing precise notation matched to the reference implementation. Section ?? develops the inexact IPM theory, defining the relative residual η_k , analyzing its propagation through the IPM step (primal feasibility, dual feasibility, and duality measure), stating the polynomial convergence theorem, and discussing the feasible vs. infeasible distinction. Section ?? presents the block-partition structure of $\mathbf{M}_k$, the 2×2 block Cholesky with Schur complement, the block-diagonal preconditioner, the three-level precision strategy, and the `LowPrecisionCholeskySolver` and `BlockJacobiPCGSolver` algorithms. Section ?? introduces the μ -driven tolerance schedule, the complete `AdaptiveRefinementSolver` algorithm, a detailed design analysis, and the connection to QIPM. Section ?? reports the numerical results. Section ?? concludes with a summary and directions for future work.

2 Preliminaries

2.1 Standard-form linear programming

We consider the primal linear program in standard form and its Lagrangian dual:

$$\min_{\mathbf{x} \in \mathbb{R}^n} \mathbf{c}^T \mathbf{x} \quad \text{subject to} \quad \mathbf{A}\mathbf{x} = \mathbf{b}, \quad \mathbf{x} \geq 0, \quad (3)$$

$$\max_{\mathbf{y} \in \mathbb{R}^m, \mathbf{s} \in \mathbb{R}^n} \mathbf{b}^T \mathbf{y} \quad \text{subject to} \quad \mathbf{A}^T \mathbf{y} + \mathbf{s} = \mathbf{c}, \quad \mathbf{s} \geq 0. \quad (4)$$

Here $\mathbf{A} \in \mathbb{R}^{m \times n}$ is the constraint matrix (typically $m \leq n$), $\mathbf{b} \in \mathbb{R}^m$ is the right-hand side vector, and $\mathbf{c} \in \mathbb{R}^n$ is the cost vector. The component-wise inequalities $\mathbf{x} \geq 0$ and $\mathbf{s} \geq 0$ define the non-negative orthant. Problem (??) is called the *primal* LP and Problem (??) is the *dual* LP; the variable $\mathbf{x}$ is the primal solution, $\mathbf{y}$ is the dual solution (or Lagrange multiplier for the equality constraint $\mathbf{A}\mathbf{x} = \mathbf{b}$), and $\mathbf{s}$ is the dual slack variable.

The first-order necessary and sufficient optimality conditions for the pair (??)–(??) are the Karush–Kuhn–Tucker (KKT) system

$$\begin{aligned} \mathbf{A}\mathbf{x} &= \mathbf{b}, \\ \mathbf{A}^T \mathbf{y} + \mathbf{s} &= \mathbf{c}, \\ \mathbf{x} \circ \mathbf{s} &= \mathbf{0}, \\ \mathbf{x} &\geq \mathbf{0}, \quad \mathbf{s} \geq \mathbf{0}, \end{aligned} \quad (5)$$

where $\mathbf{x} \circ \mathbf{s} \in \mathbb{R}^n$ denotes the Hadamard (entrywise) product, i.e., $(\mathbf{x} \circ \mathbf{s})_i = x_i s_i$ for $i = 1, \dots, n$, and $\mathbf{0}$ is the vector of all zeros. The first equation of (??) is primal feasibility; the second is dual feasibility; and the third is the complementarity condition, which requires $x_i s_i = 0$ for every i . Because $\mathbf{x} \geq 0$ and $\mathbf{s} \geq 0$, the complementarity condition is equivalent to the scalar equation $\mathbf{x}^T \mathbf{s} = 0$, which states that the duality gap is zero at optimality.

2.2 The central path and the duality measure

Primal-dual interior point methods replace the non-smooth complementarity condition of (??) with the *perturbed* condition

$$\mathbf{x} \circ \mathbf{s} = \mu \mathbf{1}_n, \quad (6)$$

where $\mu > 0$ is a centering parameter and $\mathbf{1}_n$ is the vector of all ones in $\mathbb{R}^n$. For each fixed $\mu > 0$, the perturbed KKT system

$$\mathbf{A}\mathbf{x} = \mathbf{b}, \quad \mathbf{A}^T \mathbf{y} + \mathbf{s} = \mathbf{c}, \quad \mathbf{x} \circ \mathbf{s} = \mu \mathbf{1}_n, \quad \mathbf{x} > \mathbf{0}, \quad \mathbf{s} > \mathbf{0}, \quad (7)$$

has a unique solution $(\mathbf{x}(\mu), \mathbf{y}(\mu), \mathbf{s}(\mu))$ under mild assumptions (e.g., $\mathbf{A}$ has full row rank and a strictly feasible interior point exists). The set

$$\mathcal{C} := \{(\mathbf{x}(\mu), \mathbf{y}(\mu), \mathbf{s}(\mu)) \mid \mu > 0\} \quad (8)$$

is called the *central path*. As $\mu \rightarrow 0$, the central path converges to a solution of the original KKT system (??).

Taking the inner product of both sides of (??) with $\mathbf{1}_n$ yields

$$\mathbf{x}^T \mathbf{s} = \sum_{i=1}^n x_i s_i = n\mu, \quad (9)$$

so that the quantity

$$\mu := \frac{\mathbf{x}^T \mathbf{s}}{n} \quad (10)$$

is the average complementarity product at the current iterate. We refer to μ as the *duality measure*; it equals the normalized duality gap and is the natural measure of progress toward optimality in a primal-dual IPM. In the code, μ is computed exactly as in (??): `mu = float(np.dot(x, s)/n)`.

2.3 Newton system and the normal equations

At each iteration of a primal-dual IPM, we linearize the perturbed KKT system (??) around the current iterate $(\mathbf{x}_k, \mathbf{y}_k, \mathbf{s}_k)$ with $\mathbf{x}_k > 0$, $\mathbf{s}_k > 0$, and compute a search direction $(\Delta \mathbf{x}_k, \Delta \mathbf{y}_k, \Delta \mathbf{s}_k)$. Linearizing (??) yields the third equation

$${}_k \Delta \mathbf{x}_k + {}_k \Delta \mathbf{s}_k = \sigma_k \mu_k \mathbf{1}_n - {}_k \mathbf{s}_k, \quad (11)$$

where ${}_k := \text{diag}(\mathbf{x}_k)$, ${}_k := \text{diag}(\mathbf{s}_k)$, $\mu_k := \mathbf{x}_k^T \mathbf{s}_k / n$, and $\sigma_k \in [0, 1]$ is the *centering parameter*. Setting $\sigma_k = 1$ gives a pure centering direction (toward the point $(\mathbf{x}(\mu_k), \mathbf{y}(\mu_k), \mathbf{s}(\mu_k))$ on the central path), while $\sigma_k = 0$ gives the pure affine-scaling direction (toward optimality, ignoring centrality). The full linearized system is the $(2n + m) \times (2n + m)$ *augmented system*

$$\begin{bmatrix} \mathbf{0} & \mathbf{A}^T & \mathbf{I} \\ \mathbf{A} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & {}_k \end{bmatrix} \begin{bmatrix} \Delta \mathbf{x}_k \\ \Delta \mathbf{y}_k \\ \Delta \mathbf{s}_k \end{bmatrix} = \begin{bmatrix} \mathbf{c} - \mathbf{A}^T \mathbf{y}_k - \mathbf{s}_k \\ \mathbf{b} - \mathbf{A} \mathbf{x}_k \\ \sigma_k \mu_k \mathbf{1}_n - {}_k \mathbf{s}_k \end{bmatrix}. \quad (12)$$

Eliminating $\Delta \mathbf{s}_k$ via the third block row gives $\Delta \mathbf{s}_k = \sigma_k \mu_k {}_k^{-1} \mathbf{1}_n - {}_k^{-1} {}_k \Delta \mathbf{x}_k$. Substituting into the first two block rows and eliminating $\Delta \mathbf{x}_k$ reduces the system to the $m \times m$ *normal equations*

$$\boxed{\mathbf{M}_k \Delta \mathbf{y}_k = {}_k}, \quad \mathbf{M}_k := \mathbf{A} \mathbf{D}_k^2 \mathbf{A}^T, \quad (13)$$

where the *scaling matrix* is

$$\mathbf{D}_k := \text{diag}(d_1^{(k)}, \dots, d_n^{(k)}), \quad d_i^{(k)} := \sqrt{\frac{x_i^{(k)}}{s_i^{(k)}}}, \quad i = 1, \dots, n, \quad (14)$$

and the right-hand side ${}_k \in \mathbb{R}^m$ is assembled from the residuals and the centering term, as detailed in Section ?? below. In the codebase, $\mathbf{M}_k$ is formed by the function `compute_normal_equation`, which computes `A_scaled = A * D_diag[None, :]` followed by `A_scaled @ A_scaled.T`.

The normal-equations matrix $\mathbf{M}_k$ is symmetric positive definite (SPD) whenever $\mathbf{x}_k > 0$, $\mathbf{s}_k > 0$, and $\mathbf{A}$ has full row rank. This SPD property is the key advantage of the normal-equations formulation: it allows the use of Cholesky factorization, conjugate gradient, and other methods specialized for SPD systems.

Once $\Delta \mathbf{y}_k$ has been computed (exactly or approximately, as discussed in Section ??), the primal and dual slack directions are recovered via

$$\Delta \mathbf{x}_k = -\mathbf{x}_k + \frac{\sigma_k \mu_k}{\mathbf{s}_k} - \frac{\mathbf{x}_k}{\mathbf{s}_k} \circ (\mathbf{c} - \mathbf{A}^T \mathbf{y}_k - \mathbf{s}_k) + \frac{\mathbf{x}_k}{\mathbf{s}_k} \circ (\mathbf{A}^T \Delta \mathbf{y}_k) - \frac{\Delta \mathbf{x}_k^{\text{aff}} \circ \Delta \mathbf{s}_k^{\text{aff}}}{\mathbf{s}_k}, \quad (15)$$

$$\Delta \mathbf{s}_k = (\mathbf{c} - \mathbf{A}^T \mathbf{y}_k - \mathbf{s}_k) - \mathbf{A}^T \Delta \mathbf{y}_k, \quad (16)$$

where all divisions and multiplications involving vectors are component-wise, and the final term in (??) is the second-order Mehrotra correction (see Section ??).

2.4 The Mehrotra predictor-corrector algorithm

We adopt the widely used Mehrotra [?] predictor-corrector scheme, which is the workhorse of virtually all practical IPM implementations. The algorithm computes, at each iteration, two directions using the *same* coefficient matrix $\mathbf{M}_k$ but different right-hand sides.

2.4.1 Step 1: Predictor (affine-scaling) direction

Set $\sigma = 0$ (no centering) and omit the second-order correction. Define the dual residual at the current iterate

$$d^{(k)} := \mathbf{c} - \mathbf{A}^T \mathbf{y}_k - \mathbf{s}_k. \quad (17)$$

The predictor right-hand side (in $\mathbb{R}^m$) is

$$\mathbf{x}_k^{\text{aff}} := \mathbf{b} + \mathbf{A} \left(\frac{\mathbf{x}_k}{\mathbf{s}_k} \circ_d d^{(k)} \right). \quad (18)$$

Solve the normal equations (exactly or inexactly) to obtain $\Delta \mathbf{y}_k^{\text{aff}}$, and recover the affine primal and dual slack directions $\Delta \mathbf{x}_k^{\text{aff}}$ and $\Delta \mathbf{s}_k^{\text{aff}}$ via (??)–(??) with $\sigma = 0$ and no correction term.

2.4.2 Step 2: Fraction-to-boundary step lengths

The step length to the boundary of the non-negative orthant is determined by the *fraction-to-boundary* rule. For a current iterate > 0 and a direction Δ , the maximum step that preserves non-negativity is

$$\alpha_{\max}(\Delta) := \min \left\{ -\frac{v_i}{\Delta v_i} \mid i \in \{1, \dots, n\}, \Delta v_i < 0 \right\}, \quad (19)$$

with $\alpha_{\max} := \tau$ if no component of Δ is negative. To keep the iterates strictly positive and bounded away from the boundary (which would cause the scaling matrix $\mathbf{D}_k$ to blow up), the actual step length is taken as

$$\alpha(\Delta) := \min \{ \tau \cdot \alpha_{\max}(\Delta), \tau \}, \quad (20)$$

where $\tau \in (0, 1)$ is the fraction-to-boundary parameter (typically $\tau = 0.99$ or 0.9995). The step length for the dual variable $\mathbf{y}$ is taken equal to that of the dual slack $\mathbf{s}$, since $\mathbf{y}$ is unconstrained in sign. These step lengths are computed for the affine direction to obtain $\alpha_p^{\text{aff}} := \alpha(\mathbf{x}_k, \Delta \mathbf{x}_k^{\text{aff}})$ and $\alpha_d^{\text{aff}} := \alpha(\mathbf{s}_k, \Delta \mathbf{s}_k^{\text{aff}})$.

2.4.3 Step 3: Centering parameter

Let

$$\mu_k^{\text{aff}} := \frac{1}{n} (\mathbf{x}_k + \alpha_p^{\text{aff}} \Delta \mathbf{x}_k^{\text{aff}})^T (\mathbf{s}_k + \alpha_d^{\text{aff}} \Delta \mathbf{s}_k^{\text{aff}}) \quad (21)$$

be the duality measure that would result from a full step along the affine direction using the computed step lengths. The ratio μ_k^{aff}/μ_k measures how much progress the pure affine direction makes. Mehrotra's heuristic sets the centering parameter adaptively as

$$\sigma_k := \left(\frac{\mu_k^{\text{aff}}}{\mu_k} \right)^3, \quad (22)$$

with the safeguard $\sigma_k := 0$ when $\mu_k \leq 10^{-16}$. The exponent 3 is standard practice: when the affine direction makes substantial progress ($\mu_k^{\text{aff}} \ll \mu_k$), σ_k becomes very small, biasing the corrector step toward optimality; conversely, when the affine direction makes little progress, $\sigma_k \approx 1$ and the corrector step emphasizes centrality.

2.4.4 Step 4: Corrector (combined) direction

The corrector right-hand side assembles three components:

$$\mathbf{r}_k^{\text{corr},1} := \mathbf{b} + \mathbf{A} \left(\frac{\mathbf{x}_k}{\mathbf{s}_k} \circ_d^{(k)} \right), \quad (23)$$

$$\mathbf{r}_k^{\text{corr},2} := \sigma_k \mu_k \mathbf{A} \left(\frac{\mathbf{1}_n}{\mathbf{s}_k} \right), \quad (24)$$

$$\mathbf{r}_k^{\text{corr},3} := \mathbf{A} \left(\frac{\Delta \mathbf{x}_k^{\text{aff}} \circ \Delta \mathbf{s}_k^{\text{aff}}}{\mathbf{s}_k} \right), \quad (25)$$

where (??) is the linearized residual term (identical in form to the predictor RHS (??)), (??) is the centering term (which vanishes when $\sigma_k \rightarrow 0$), and (??) is Mehrotra's second-order nonlinear correction that uses the Hadamard product of the affine directions to approximate the quadratic term in the Taylor expansion of the complementarity condition. The full corrector right-hand side is

$$\mathbf{r}_k^{\text{corr}} := \mathbf{r}_k^{\text{corr},1} - \mathbf{r}_k^{\text{corr},2} + \mathbf{r}_k^{\text{corr},3}. \quad (26)$$

The normal equations $\mathbf{M}_k \Delta \mathbf{y}_k = \mathbf{r}_k^{\text{corr}}$ are solved with the *same* coefficient matrix $\mathbf{M}_k$ as the predictor step (which is critical for efficiency: one factorization serves two solves). The primal and dual directions are then recovered via (??)–(??) with $\sigma = \sigma_k$ and the correction term included.

2.4.5 Step 5: Step length and update

The step lengths for the combined (corrector) direction are computed via (??):

$$\alpha_p^{(k)} := \alpha(\mathbf{x}_k, \Delta \mathbf{x}_k), \quad \alpha_d^{(k)} := \alpha(\mathbf{s}_k, \Delta \mathbf{s}_k). \quad (27)$$

The iterate is updated as

$$\begin{aligned} \mathbf{x}_{k+1} &:= \mathbf{x}_k + \alpha_p^{(k)} \Delta \mathbf{x}_k, \\ \mathbf{s}_{k+1} &:= \mathbf{s}_k + \alpha_d^{(k)} \Delta \mathbf{s}_k, \\ \mathbf{y}_{k+1} &:= \mathbf{y}_k + \alpha_d^{(k)} \Delta \mathbf{y}_k. \end{aligned} \quad (28)$$

Notice that the dual variable $\mathbf{y}$ steps with $\alpha_d^{(k)}$ (the same step length as the dual slack), which is standard practice since both live in the dual space and using the smaller of α_p and α_d for $\mathbf{y}$ would be overly conservative.

2.4.6 Convergence monitoring

At each iteration, the relative primal and dual residuals are computed as

$$\rho_p^{(k)} := \frac{\|\mathbf{b} - \mathbf{A} \mathbf{x}_k\|_2}{1 + \|\mathbf{b}\|_2}, \quad (29)$$

$$\rho_d^{(k)} := \frac{\|\mathbf{c} - \mathbf{A}^T \mathbf{y}_k - \mathbf{s}_k\|_2}{1 + \|\mathbf{c}\|_2}, \quad (30)$$

where the $1 + \|\cdot\|_2$ denominators follow standard practice for scale-invariant stopping criteria [?]. The IPM is considered converged when

$$\mu_k < \varepsilon, \quad \rho_p^{(k)} < \varepsilon, \quad \rho_d^{(k)} < \varepsilon, \quad (31)$$

with a typical tolerance of $\varepsilon = 10^{-8}$.

2.5 Conditioning of the normal equations

A fundamental numerical difficulty with the normal-equations formulation is that the spectral condition number of $\mathbf{M}_k$ can be dramatically worse than that of the original constraint matrix $\mathbf{A}$. From the definition $\mathbf{M}_k = \mathbf{A}\mathbf{D}_k^2\mathbf{A}^T$ and the submultiplicativity of the spectral norm,

$$\kappa(\mathbf{M}_k) = \kappa(\mathbf{A}\mathbf{D}_k^2\mathbf{A}^T) \leq \kappa(\mathbf{A})^2 \cdot \left(\frac{\max_i d_i^{(k)}}{\min_i d_i^{(k)}} \right)^2, \quad (32)$$

where $\kappa(\cdot) := \sigma_{\max}(\cdot)/\sigma_{\min}(\cdot)$ is the ratio of largest to smallest singular value. The factor $(\max d_i / \min d_i)^2$ reflects the spread of the diagonal scaling entries. In the early iterations of an IPM, the entries of $\mathbf{x}_k$ and $\mathbf{s}_k$ are typically of comparable magnitude, so $d_i^{(k)} = \sqrt{x_i/s_i}$ is moderate and $\mathbf{M}_k$ is well-conditioned. However, as the IPM converges and $\mu_k \rightarrow 0$, the primal and dual variables separate: some x_i approach zero (active constraints) while the corresponding s_i remain bounded away from zero (or vice versa). Consequently, the ratio $\max_i d_i / \min_i d_i$ can span many orders of magnitude—often 10^8 or larger in the final iterations—and $\kappa(\mathbf{M}_k)$ can exceed 10^{16} , the reciprocal of machine epsilon in FP64.

This progressive ill-conditioning is the central numerical challenge that any low-precision or inexact linear solver must confront inside an IPM. It motivates the adaptive precision schedule developed in Section ?? and the fallback mechanisms embedded in the solvers (see Section ??).

3 Inexact IPM Theory

In an exact IPM, the normal equations (??) are solved to machine precision at each iteration. In an *inexact* IPM, we permit an approximate solution, trading linear-solver accuracy for computational speed. This section formalizes the notion of inexactness, analyzes how the linear-solve error propagates through the IPM step, and states the convergence guarantees that underpin the adaptive scheduling strategy of Section ??.

3.1 Inexact Newton directions

Let $\widehat{\Delta\mathbf{y}}_k$ denote the *computed* (approximate) solution of the normal equations at iteration k . We write

$$\mathbf{M}_k \widehat{\Delta\mathbf{y}}_k = \mathbf{r}_k + \mathbf{lin}_k, \quad (33)$$

where

$$\mathbf{lin}_k := \mathbf{M}_k \widehat{\Delta\mathbf{y}}_k - \mathbf{r}_k \quad (34)$$

is the *linear-system residual*. The exact Newton direction $\Delta\mathbf{y}_k^*$ satisfies $\mathbf{M}_k \Delta\mathbf{y}_k^* = \mathbf{r}_k$, so $\mathbf{lin}_k = \mathbf{M}_k(\widehat{\Delta\mathbf{y}}_k - \Delta\mathbf{y}_k^*)$.

Definition 3.1 (Relative residual). *The relative residual of the linear solve is*

$$\eta_k := \frac{\|\mathbf{lin}_k\|_2}{\|\mathbf{r}_k\|_2} \equiv \frac{\|\mathbf{M}_k \widehat{\Delta\mathbf{y}}_k - \mathbf{r}_k\|_2}{\|\mathbf{r}_k\|_2}. \quad (35)$$

When $\|\mathbf{r}_k\|_2 < 10^{-15}$ we set $\eta_k := 0$.

This is precisely the quantity tracked by the `_compute_eta` method in the codebase: the residual is evaluated in FP64 regardless of the internal precision used to compute $\widehat{\Delta \mathbf{y}}_k$, so η_k gives a faithful, precision-independent measure of backward error.

The central question of inexact IPM theory is: *how large can η_k be at iteration k while still guaranteeing that the outer IPM converges, and at what asymptotic rate?*

3.2 Error propagation through the IPM step

3.2.1 Error in the primal-dual directions

Given the approximate dual direction $\widehat{\Delta \mathbf{y}}_k$, the primal and dual slack directions are recovered via (??)–(??) (omitting the corrector terms for clarity; the analysis generalizes straightforwardly):

$$\widehat{\Delta \mathbf{x}}_k = -\mathbf{x}_k + \frac{\sigma_k \mu_k}{\mathbf{s}_k} - \frac{\mathbf{x}_k}{\mathbf{s}_k} \circ_d^{(k)} + \frac{\mathbf{x}_k}{\mathbf{s}_k} \circ (\mathbf{A}^T \widehat{\Delta \mathbf{y}}_k), \quad (36)$$

$$\widehat{\Delta \mathbf{s}}_k = \circ_d^{(k)} - \mathbf{A}^T \widehat{\Delta \mathbf{y}}_k, \quad (37)$$

where $\circ_d^{(k)} = \mathbf{c} - \mathbf{A}^T \mathbf{y}_k - \mathbf{s}_k$ is the dual residual at the current iterate. Let $\Delta \mathbf{x}_k^*$ and $\Delta \mathbf{s}_k^*$ be the exact directions that would be obtained by using $\Delta \mathbf{y}_k^*$ in the same formulas. Subtracting the exact from the inexact recovery yields the *direction errors*:

$$\widehat{\Delta \mathbf{x}}_k - \Delta \mathbf{x}_k^* = \circ_k^{1/2} \circ_k^{-1/2} \circ \mathbf{A}^T (\widehat{\Delta \mathbf{y}}_k - \Delta \mathbf{y}_k^*) = \mathbf{D}_k \circ \mathbf{A}^T (\widehat{\Delta \mathbf{y}}_k - \Delta \mathbf{y}_k^*), \quad (38)$$

$$\widehat{\Delta \mathbf{s}}_k - \Delta \mathbf{s}_k^* = -\mathbf{A}^T (\widehat{\Delta \mathbf{y}}_k - \Delta \mathbf{y}_k^*). \quad (39)$$

The error in the dual direction propagates to the primal direction *scaled by the factor $\mathbf{D}_k$* , which amplifies the error precisely when the scaling matrix is ill-conditioned (i.e., in late IPM iterations).

3.2.2 Propagation to primal feasibility

The primal residual after taking a step of length α in the inexact direction is

$$\begin{aligned} \circ_p^{(k+1)} &= \mathbf{b} - \mathbf{A}(\mathbf{x}_k + \alpha_k \widehat{\Delta \mathbf{x}}_k) \\ &= \mathbf{b} - \mathbf{A}\mathbf{x}_k - \alpha_k \mathbf{A}\Delta \mathbf{x}_k^* - \alpha_k \mathbf{A}(\widehat{\Delta \mathbf{x}}_k - \Delta \mathbf{x}_k^*) \\ &= (1 - \alpha_k)(\mathbf{b} - \mathbf{A}\mathbf{x}_k) + \alpha_k \mathbf{A}\mathbf{D}_k^2 \mathbf{A}^T (\widehat{\Delta \mathbf{y}}_k - \Delta \mathbf{y}_k^*), \end{aligned} \quad (40)$$

where we used $\mathbf{A}\Delta \mathbf{x}_k^* = \mathbf{b} - \mathbf{A}\mathbf{x}_k$ (the exact direction restores primal feasibility exactly in one full step) and (??) to express the direction error in terms of $\widehat{\Delta \mathbf{y}}_k - \Delta \mathbf{y}_k^*$. Recalling that $\mathbf{M}_k = \mathbf{A}\mathbf{D}_k^2 \mathbf{A}^T$ and $\circ_k^{\text{lin}} = \mathbf{M}_k(\widehat{\Delta \mathbf{y}}_k - \Delta \mathbf{y}_k^*)$, we obtain the compact relation

$$\boxed{\circ_p^{(k+1)} = (1 - \alpha_k) \circ_p^{(k)} + \alpha_k \circ_k^{\text{lin}}}. \quad (41)$$

Equation (??) reveals a fundamental tension: the step length α_k simultaneously reduces the current primal residual (the first term) and injects the linear-solve error (the second term). If $\|\circ_k^{\text{lin}}\|$ is large relative to $\|\circ_p^{(k)}\|$, the primal residual may *grow* rather than shrink. This phenomenon is observed in practice: our implementation detects the condition

$$\mu_k < 10^{-6} \text{ and } \rho_p^{(k)} > 10^{-3} \quad (42)$$

as an indicator that the linear-solve error has irreparably damaged primal feasibility, and the IPM is terminated.

3.2.3 Propagation to dual feasibility

A similar analysis yields the dual residual after the step:

$$\mathbf{d}^{(k+1)} = (1 - \alpha_k) \mathbf{d}^{(k)} + \alpha_k \mathbf{A}^T (\widehat{\Delta \mathbf{y}}_k - \Delta \mathbf{y}_k^*). \quad (43)$$

Comparing with (??), the dual residual receives the *unscaled* direction error, while the primal residual receives it premultiplied by $\mathbf{A} \mathbf{D}_k^2 \mathbf{A}^T = \mathbf{M}_k$. Since $\mathbf{M}_k$ can be highly ill-conditioned (see Section ??), the primal residual is typically more sensitive to inexactness than the dual residual.

3.2.4 Propagation to the duality measure

The duality measure after the step is

$$\mu_{k+1} = \frac{1}{n} (\mathbf{x}_k + \alpha_k \widehat{\Delta \mathbf{x}}_k)^T (\mathbf{s}_k + \alpha_k \widehat{\Delta \mathbf{s}}_k). \quad (44)$$

Expanding and using $\mathbf{x}_k^T \mathbf{s}_k = n\mu_k$, one can show that

$$\mu_{k+1} = (1 - \alpha_k(1 - \sigma_k))\mu_k + \mathcal{O}(\alpha_k^2 \|\cdot\|^2) + \mathcal{O}(\alpha_k \eta_k \|\cdot\|), \quad (45)$$

where the final $\mathcal{O}(\alpha_k \eta_k \|\cdot\|)$ term captures the effect of the inexactness. This term must be dominated by the reduction in μ for convergence to be maintained.

3.3 Convergence theorem

The convergence of inexact IPMs is governed by how the allowable residual η_k scales with the duality measure μ_k . We state the central result (without full proof; see [?, ?] for detailed analyses).

Assumption 3.1 (Interior starting point). *The initial iterate $(\mathbf{x}_0, \mathbf{y}_0, \mathbf{s}_0)$ is strictly feasible ($\mathbf{x}_0 > 0$, $\mathbf{s}_0 > 0$) with $\mathbf{A}\mathbf{x}_0 = \mathbf{b}$ and $\mathbf{A}^T \mathbf{y}_0 + \mathbf{s}_0 = \mathbf{c}$, and lies in a suitable neighborhood $\mathcal{N}_2(\beta)$ of the central path:*

$$\|\mathbf{x}_0 \circ \mathbf{s}_0 - \mu_0 \mathbf{1}_n\|_2 \leq \beta \mu_0, \quad \beta \in (0, 1). \quad (46)$$

Assumption 3.2 (Full rank). *The constraint matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ has full row rank, so that $\mathbf{M}_k = \mathbf{A} \mathbf{D}_k^2 \mathbf{A}^T$ is strictly positive definite for all $\mathbf{x}_k, \mathbf{s}_k > 0$.*

Theorem 3.1 (Polynomial convergence of inexact IPM). *Let Assumptions ?? and ?? hold. Suppose that at each iteration k of the Mehrotra predictor-corrector IPM, the linear solve satisfies*

$$\boxed{\eta_k \leq \bar{c} \mu_k^\alpha}, \quad (47)$$

for constants $\bar{c} > 0$ and $\alpha \in (0, 1]$ independent of k . Then there exists a constant $\delta > 0$ such that

$$\mu_{k+1} \leq \left(1 - \frac{\delta}{\sqrt{n}}\right) \mu_k, \quad (48)$$

and the method converges to a point with $\mu \leq \varepsilon$ in at most

$$K = \mathcal{O}(\sqrt{n} \log(1/\varepsilon)) \quad (49)$$

iterations.

Proof sketch. The proof adapts the standard analysis of polynomial IPM convergence (see, e.g., [?, ?]) by tracking the effect of the inexactness residual through the predictor and corrector steps separately.

Predictor step. The affine direction with $\sigma = 0$ reduces μ by a factor that, in the exact case, is $(1 - \delta/\sqrt{n})$. With inexactness, the reduction is attenuated by an $\mathcal{O}(\eta_k)$ term. Condition (??) ensures that $\eta_k \leq \bar{c}\mu_k$ (when $\alpha \geq 1$) or $\eta_k \leq \bar{c}\mu_k^\alpha$ (when $\alpha < 1$), so the inexactness budget is always small relative to the current duality gap, preserving the polynomial reduction rate.

Corrector step. The centering component (with σ_k given by (??)) restores proximity to the central path. The inexactness adds a perturbation that is $\mathcal{O}(\eta_k^2)$ in the centrality measure (??), which is dominated by the linear term when η_k is sufficiently small.

Asymptotic behavior. When $\mu_k \ll 1$ and α is chosen appropriately, η_k shrinks rapidly with μ_k . If $\alpha \geq 1$, then η_k shrinks at least linearly with μ_k , and the quadratic convergence of Mehrotra’s method (in the final iterations) is preserved. If $\alpha < 1$, the inexactness decays more slowly than μ_k , and the asymptotic rate degrades to linear, though the polynomial complexity bound (??) still holds.

Feasibility drift. Equation (??) shows that the primal residual evolves as $p^{(k+1)} = (1 - \alpha_k)p^{(k)} + \alpha_k \text{lin}_k$. Under (??), the injected error lin_k is bounded by $\bar{c}\mu_k^\alpha \|k\|$, which is $\mathcal{O}(\mu_k^\alpha)$ when the RHS k is of moderate size. Since μ_k itself decreases geometrically by (??), the cumulative feasibility drift over K iterations is bounded, and the final primal infeasibility is $\mathcal{O}(\varepsilon^\alpha)$. $\square$

Remark 3.1. The condition $\eta_k \leq \bar{c}\mu_k^\alpha$ subsumes several special cases from the literature. Bellavia [?] uses $\eta_k \leq \gamma\mu_k$ (i.e., $\alpha = 1$, $\bar{c} = \gamma$), while Monteiro and O’Neal [?] consider $\eta_k \leq \gamma \min\{\mu_k, 1\}$, which is equivalent to $\alpha = 1$ with a cap. Our adaptive schedule (Section ??) uses a piecewise-constant approximation to a μ^α envelope, which is easier to implement and tune in practice.

3.4 Feasible vs. infeasible inexact IPM

Equation (??) highlights a critical distinction between two classes of inexact IPM formulations:

Definition 3.2 (Inexact feasible IPM). *An IPM is called inexact feasible if the Newton direction is constructed so that primal and dual feasibility are preserved exactly, even when the linear solve is approximate. This requires that the inexactness residual lin_k lie in the null space of $\mathbf{A}$ (for primal feasibility) or that the direction recovery formulas be modified to project out the error [?].*

Definition 3.3 (Inexact infeasible IPM). *An IPM is called inexact infeasible if no special mechanism is used to preserve feasibility, and the residuals evolve according to (??)–(??). This is the natural setting for the normal-equations formulation with an approximate $\widehat{\Delta\mathbf{y}}_k$.*

The prototype implemented in this paper is an *inexact infeasible* IPM. The practical implications depend on the phase of the solve. In the early iterations, μ_k is large, the normal-equations matrix is well-conditioned, and the linear-solve error η_k is small (either because the low-precision factorization is accurate, or because iterative refinement converges quickly); the feasibility drift (??) is negligible. In the mid iterations, as μ_k decreases, $\kappa(\mathbf{M}_k)$ grows and the low-precision factorization loses accuracy; the adaptive schedule tightens η_{target} , triggering more refinement iterations to keep η_k within bounds; primal feasibility may degrade moderately, but the IPM corrector step (with $\sigma_k > 0$) restores centrality and helps contain the drift. In the late iterations, when $\mu_k < 10^{-6}$, even small linear-solve errors can translate into large primal residuals because $\mathbf{D}_k$ amplifies them via (??); at this stage, the adaptive schedule demands near-exact solves ($\eta_{\text{target}} = 10^{-10}$), and the fallback to FP64 Cholesky ensures that the final solution accuracy is not compromised. Finally, the

code monitors the condition (??) and terminates early if the inexactness has pushed the IPM into an irrecoverable infeasible state: this is a pragmatic safeguard that avoids wasting computation on iterates that can no longer converge and provides diagnostic feedback for tuning the tolerance schedule.

The inexact infeasible formulation is both simpler to implement and more natural for the normal-equations framework than a feasibility-preserving alternative (e.g., via null-space projection or augmented KKT systems). The price is the need for the heuristic safeguards described above; the numerical results in Section ?? confirm that these safeguards are effective in practice.

Remark 3.2 (Connection to QIPM). *Quantum interior point methods [?] face an analogous trade-off: the quantum linear system algorithm (QLSA) outputs an approximate solution whose error must be absorbed by the outer IPM. The QIPM literature has therefore developed a detailed theory of inexact feasible IPMs, where the Newton reformulation (often via null-space methods) preserves feasibility regardless of linear-solve accuracy. Our work can be viewed as a classical analogue: we use low-precision GPU arithmetic in place of QLSA noise, and iterative refinement in place of quantum state tomography, but the mathematical structure of inexactness is similar. This connection suggests that feasibility-preserving reformulations developed for QIPM could also benefit classical GPU-accelerated IPMs, and vice versa.*

4 Multi-GPU Block Cholesky Preconditioner

The normal-equations matrix $\mathbf{M}_k = \mathbf{A}\mathbf{D}_k^2\mathbf{A}^T$ is an $m \times m$ symmetric positive definite matrix. For large m (e.g., $m \geq 10^4$), a full dense Cholesky factorization $\mathbf{M}_k = \mathbf{L}\mathbf{L}^T$ costs $\mathcal{O}(m^3)$ operations and requires $\mathcal{O}(m^2)$ storage, which may exceed the memory of a single GPU. This section develops a *block-diagonal Cholesky preconditioner* that distributes the factorization across multiple GPUs and serves as the computational core of all the inexact solvers described in this paper.

4.1 Block partition of the normal-equations matrix

We partition the index set $\{1, \dots, m\}$ into p contiguous blocks of sizes $m_1, \dots, m_p$ (with $\sum_{i=1}^p m_i = m$). The normal-equations matrix inherits the corresponding block structure:

$$\mathbf{M}_k = \begin{bmatrix} \mathbf{M}_{11} & \mathbf{M}_{12} & \cdots & \mathbf{M}_{1p} \\ \mathbf{M}_{21} & \mathbf{M}_{22} & \cdots & \mathbf{M}_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{M}_{p1} & \mathbf{M}_{p2} & \cdots & \mathbf{M}_{pp} \end{bmatrix}, \quad \mathbf{M}_{ij} \in \mathbb{R}^{m_i \times m_j}, \quad (50)$$

where $\mathbf{M}_{ij} = \mathbf{A}_i\mathbf{D}_k^2\mathbf{A}_j^T$ and $\mathbf{A}_i \in \mathbb{R}^{m_i \times n}$ is the i -th row-block of the constraint matrix. Because $\mathbf{M}_k$ is symmetric, $\mathbf{M}_{ji} = \mathbf{M}_{ij}^T$.

In our multi-GPU setting, the i -th diagonal block $\mathbf{M}_{ii}$ (and the corresponding row-block $\mathbf{A}_i$) is assigned to GPU i . The block sizes m_i are chosen to balance GPU memory utilization; a simple equal-split strategy sets $m_i \approx m/p$ for all i , as implemented in `_build_block_diagonal_preconditioner` by the slicing `start = i * m // p; end = (i+1) * m // p`.

4.2 Block Cholesky factorization with Schur complement

To understand the relationship between a full Cholesky factorization and the block-diagonal approximation, it is instructive to examine the $p = 2$ case in detail. Write

$$\mathbf{M}_k = \begin{bmatrix} \mathbf{M}_{11} & \mathbf{M}_{12} \\ \mathbf{M}_{21} & \mathbf{M}_{22} \end{bmatrix}, \quad \mathbf{M}_{11} \in \mathbb{R}^{m_1 \times m_1}, \mathbf{M}_{22} \in \mathbb{R}^{m_2 \times m_2}, \quad (51)$$

with $\mathbf{M}_{12}^T = \mathbf{M}_{21}$. The *block Cholesky factorization* (generalizing the standard block LDL^T decomposition to the SPD case) is

$$\mathbf{M}_k = \begin{bmatrix} \mathbf{11} & \mathbf{0} \\ \mathbf{M}_{21\mathbf{11}}^{-T} & \mathbf{22} \end{bmatrix} \begin{bmatrix} \mathbf{11}^T & \mathbf{11}^{-1}\mathbf{M}_{12} \\ \mathbf{0} & \mathbf{22}^T \end{bmatrix}, \quad (52)$$

where

$$\mathbf{M}_{11} = \mathbf{11} \mathbf{11}^T, \quad (53)$$

$$\mathbf{22} := \mathbf{M}_{22} - \mathbf{M}_{21}\mathbf{M}_{11}^{-1}\mathbf{M}_{12} = \mathbf{22} \mathbf{22}^T. \quad (54)$$

The matrix $\mathbf{22}$ is the *Schur complement* of $\mathbf{M}_{11}$ in $\mathbf{M}_k$. It is SPD whenever $\mathbf{M}_k$ is SPD. Equation (??) reveals the essential difficulty of distributed block factorization: even if diagonal blocks $\mathbf{M}_{11}$ and $\mathbf{M}_{22}$ are factorized independently on separate GPUs, the second block's factor $\mathbf{22}$ requires the Schur complement update $\mathbf{22} = \mathbf{M}_{22} - \mathbf{M}_{21}\mathbf{M}_{11}^{-1}\mathbf{M}_{12}$, which couples the two GPUs through the off-diagonal block $\mathbf{M}_{21}$. Computing $\mathbf{M}_{21}\mathbf{M}_{11}^{-1}\mathbf{M}_{12}$ involves solving triangular systems with $\mathbf{11}$ and then a matrix-matrix multiplication, both of which require communication when $\mathbf{M}_{11}$ and $\mathbf{M}_{22}$ reside on different devices.

For $p > 2$, the block Cholesky proceeds recursively: after factorizing $\mathbf{M}_{11} = \mathbf{11} \mathbf{11}^T$, the trailing $(p-1) \times (p-1)$ submatrix is updated by the Schur complement of $\mathbf{M}_{11}$, and the process repeats. The communication pattern is all-to-all in the worst case, and the total communication volume grows with p .

4.3 Block-diagonal preconditioner: trading exactness for parallelism

Rather than attempting a full distributed block Cholesky (which would require substantial inter-GPU communication for the Schur complement updates), we construct a *block-diagonal preconditioner*:

$$\boxed{\mathbf{P}_k := \text{blkdiag}(\mathbf{M}_{11}, \mathbf{M}_{22}, \dots, \mathbf{M}_{pp})}, \quad (55)$$

where each diagonal block is factorized *independently*:

$$\mathbf{M}_{ii} = \tilde{\mathbf{i}}_i \tilde{\mathbf{i}}_i^T, \quad i = 1, \dots, p. \quad (56)$$

The tildes on $\tilde{\mathbf{i}}_i$ indicate that the factorization may be performed in reduced precision (see Section ??).

The preconditioner $\mathbf{P}_k$ is an *approximation* to $\mathbf{M}_k$: it coincides with $\mathbf{M}_k$ on the diagonal blocks but ignores the off-diagonal coupling. The quality of the approximation is characterized by the generalized eigenvalue problem: if λ is an eigenvalue of $\mathbf{P}_k^{-1}\mathbf{M}_k$, then the condition number of the preconditioned system governs the convergence rate of preconditioned Krylov methods. A standard bound (via block Gershgorin) is

$$\kappa(\mathbf{P}_k^{-1/2}\mathbf{M}_k\mathbf{P}_k^{-1/2}) \leq 1 + \max_{1 \leq i \leq p} \sum_{\substack{j=1 \\ j \neq i}}^p \|\mathbf{M}_{ii}^{-1/2}\mathbf{M}_{ij}\mathbf{M}_{jj}^{-1/2}\|_2, \quad (57)$$

where the sum over $j \neq i$ quantifies the *off-diagonal coupling strength*. When the off-diagonal blocks are small in norm relative to the diagonal blocks, the preconditioned operator is close to the identity and Krylov methods converge in few iterations. Conversely, when the couplings are strong (e.g., when $\mathbf{A}$ is dense with no natural block structure), the preconditioner is weaker and more inner iterations are required.

This design reflects a deliberate trade-off: we sacrifice the exactness of a full Cholesky solve for *embarrassingly parallel* factorization across p GPUs with zero inter-GPU communication during the factorization phase. The resulting error is absorbed by the inexact IPM framework of Section ?? and, when necessary, corrected by iterative refinement or Krylov acceleration.

4.4 Precision strategy

Modern GPUs exhibit dramatically different throughput across floating-point formats. For instance, the NVIDIA H100 GPU delivers approximately 34 TFLOPS in FP64, 67 TFLOPS in FP32, and up to 990 TFLOPS in TF32 (tensor-core) mode [?]. To exploit this hierarchy while protecting the convergence guarantees of the outer IPM, we adopt a *three-level* precision strategy.

At the first level, **factorization precision (FP32 / TF32)**, each diagonal block $\mathbf{M}_{ii}$ is downcast to FP32 (or TF32, which shares the FP32 exponent range with reduced mantissa precision) and factorized via dense Cholesky:

$$\mathbf{M}_{ii}^{\text{lo}} = \text{float32}(\mathbf{M}_{ii}), \quad \mathbf{M}_{ii}^{\text{lo}} \approx \tilde{\tilde{\mathbf{M}}}_{ii}^T. \quad (58)$$

The downcast uses the function `_safe_cast_f32` which clips values exceeding the FP32 range ($\approx 3.4 \times 10^{38}$) to prevent overflow. A small diagonal regularization $\rho \approx 10^{-10}$ is added before factorization to guard against singularity due to round-off. If the Cholesky factorization fails (e.g., due to an indefinite matrix in FP32), the code falls back to an eigendecomposition $\mathbf{M}_{ii}^{\text{lo}} = \mathbf{T}^T$ with eigenvalues clamped to a minimum of 10^{-12} .

At the second level, **residual computation (FP64)**, regardless of the precision used for factorization and solve, the linear-system residual is always evaluated in full FP64:

$$\mathbf{r}_k^{\text{lin}} = \mathbf{M}_k \widehat{\Delta \mathbf{y}}_k \quad (\text{computed in float64}). \quad (59)$$

This is critical: it provides a precision-independent backward error measure η_k that faithfully reflects the true linear-solve accuracy, preventing a situation where the solver believes it has converged but the direction is actually corrupted by low-precision round-off.

At the third level, **iterate update (FP64)**, the Newton direction $\widehat{\Delta \mathbf{y}}_k$ and the IPM iterates $(\mathbf{x}_{k+1}, \mathbf{y}_{k+1}, \mathbf{s}_{k+1})$ are accumulated in FP64. This ensures that the outer optimization is not limited by the precision of the linear algebra; only the per-iteration factorization benefits from hardware-accelerated low-precision arithmetic.

This three-level hierarchy is a direct adaptation of the mixed-precision iterative refinement framework of Carson and Higham [?] to the structured linear systems arising inside an IPM. The key difference is that our precision schedule is not static but adapts to the IPM’s progress through the μ -driven tolerance schedule of Section ??.

4.5 Solver 1: LowPrecisionCholeskySolver

The simplest inexact solver in our framework is the `LowPrecisionCholeskySolver`, which applies the three-level precision strategy to the *full* normal-equations matrix without block partitioning (i.e., $p = 1$). It simulates a single-GPU tensor-core Cholesky solve and serves as a minimal baseline for evaluating the effect of reduced precision alone.

Algorithm 1: `LowPrecisionCholeskySolver.solve(M, r)`

Input: SPD matrix $\mathbf{M} \in \mathbb{R}^{m \times m}$, right-hand side $\in \mathbb{R}^m$
Output: Solution $\widehat{\Delta \mathbf{y}}$, relative residual η
// Step 1: Downcast to FP32 with overflow guard
1 $\widetilde{\mathbf{M}} \leftarrow \text{_safe_cast_f32}(\mathbf{M})$; // clip to $[-10^{37}, 10^{37}]$, cast to float32
2 $\widetilde{\mathbf{r}} \leftarrow \text{_safe_cast_f32}()$;
// Step 2: Diagonal regularization
3 $\widetilde{\mathbf{M}}.\text{diag} += \text{float32}(10^{-10})$;
// Step 3: Factorization and solve in FP32
4 $(\widetilde{\mathbf{L}}, \text{lower}) \leftarrow \text{cho_factor}(\widetilde{\mathbf{M}})$;
5 $\widehat{\Delta \mathbf{y}}^{\text{lo}} \leftarrow \text{cho_solve}(\widetilde{\mathbf{r}})$;
6 $\text{LinAlgError}(\cdot) \leftarrow \text{eigh}(\widetilde{\mathbf{M}})$; // fallback: eigendecomposition
7 $\leftarrow \max(\cdot, 10^{-12})$;
8 $\widehat{\Delta \mathbf{y}}^{\text{lo}} \leftarrow ((\mathbf{r})^T \oslash)$;
// Step 4: Upcast to FP64
9 $\widehat{\Delta \mathbf{y}} \leftarrow \text{float64}(\widehat{\Delta \mathbf{y}}^{\text{lo}})$;
// Step 5: Fallback if FP32 produced NaN
10 **if** $\text{any}(\text{isnan}(\widehat{\Delta \mathbf{y}}))$ **then**
11 $\widehat{\Delta \mathbf{y}} \leftarrow \text{solve}(\mathbf{M}, \cdot)$; // FP64 direct solve
// Step 6: Compute relative residual in FP64
12 $\eta \leftarrow \|\mathbf{M}\widehat{\Delta \mathbf{y}} - \mathbf{r}\|_2 / \|\mathbf{r}\|_2$;
13 **return** $(\widehat{\Delta \mathbf{y}}, \eta)$;

Algorithm ?? reproduces the exact logic of the implementation. Several design choices merit comment. The overflow guard (`\_safe\_cast\_f32`) clips entries to $\pm 10^{37}$ to prevent Inf values in FP32, which has a maximum representable value of approximately 3.4×10^{38} ; for well-scaled IPM iterates this guard is rarely triggered, but it is essential for robustness with random problem data. The eigendecomposition fallback handles the rare case where round-off in FP32 causes $\widetilde{\mathbf{M}}$ to lose positive definiteness: rather than aborting, we compute $\widetilde{\mathbf{M}} = \text{diag}(\cdot)^T$ and solve via $\widehat{\Delta \mathbf{y}}^{\text{lo}} = \text{diag}(1/\cdot)^T \widetilde{\mathbf{r}}$, which is always well-defined with the eigenvalue clamping. The NaN fallback (Step 5 of the algorithm) catches catastrophic precision loss: if the FP32 computation produced non-finite values, we solve the original FP64 system directly; this is rarely executed in practice but ensures the solver never silently returns garbage. Finally, the residual is always evaluated in FP64 (Step 6, matching `\_compute\_eta`) so that η accurately reflects the backward error, not the forward error of the low-precision solve.

4.6 Solver 2: BlockJacobiPCGSolver

The `BlockJacobiPCGSolver` extends the block-diagonal preconditioner (??) to a full Krylov solver. It is the primary solver designed for the multi-GPU setting envisioned in this paper. The code is in `BlockJacobiPCGSolver.solve` and `\_build\_block\_diagonal\_preconditioner`.

Algorithm 2: BlockJacobiPCGSolver.solve($\mathbf{M}$, $\mathbf{r}$)

Input: SPD matrix $\mathbf{M} \in \mathbb{R}^{m \times m}$, right-hand side $\in \mathbb{R}^m$
Output: Solution $\widehat{\Delta \mathbf{y}}$, relative residual η
// Step 1: Build block-diagonal preconditioner
1 $p \leftarrow \min(\text{num_blocks}, m)$;
2 **for** $i \in \{0, \dots, p-1\}$ **do**
3 $\text{start} \leftarrow i \cdot m / p$, $\text{end} \leftarrow (i+1) \cdot m / p$;
4 $\leftarrow \mathbf{M}[\text{start}:\text{end}, \text{start}:\text{end}].\text{copy}()$;
5 $\text{.diag} += 10^{-12}$;
6 $\tilde{\mathbf{M}}_i \leftarrow \text{cho_factor}()$;
7 $\mathbf{P}^{-1} \leftarrow \text{LinearOperator}(x \mapsto \text{apply } \tilde{\mathbf{M}}_i \text{ to each slice of } x)$;
// Step 2: Preconditioned conjugate gradient
8 $\widehat{\Delta \mathbf{y}} \leftarrow \text{cg}(\mathbf{M}, \mathbf{r}, M = \mathbf{P}^{-1}, \text{rtol} = \text{cg_tol}, \text{maxiter} = \text{max_iter})$;
// Step 3: Residual in FP64
9 $\eta \leftarrow \|\mathbf{M}\widehat{\Delta \mathbf{y}} - \mathbf{r}\|_2 / \|\mathbf{r}\|_2$;
10 **return** $(\widehat{\Delta \mathbf{y}}, \eta)$;

4.6.1 Preconditioner construction

Given the SPD matrix $\mathbf{M}_k$ and the number of blocks p , the preconditioner is built as follows. The index range $[0, m)$ is partitioned into p contiguous slices $[\text{start}_i, \text{end}_i)$ of approximately equal size. For each slice i , the diagonal block $\mathbf{M}_{ii} = \mathbf{M}_k[\text{start}_i:\text{end}_i, \text{start}_i:\text{end}_i]$ is extracted, and a small diagonal shift $\mathbf{M}_{ii} \leftarrow \mathbf{M}_{ii} + 10^{-12} \mathbf{I}$ is applied for numerical stability. The Cholesky factorization $\mathbf{M}_{ii} = \tilde{\mathbf{M}}_i^T \tilde{\mathbf{M}}_i$ is computed via `scipy.linalg.cho_factor`, and the factor $\tilde{\mathbf{M}}_i$ and the slice boundaries are stored for subsequent solves.

The preconditioner operator $P = \text{LinearOperator}(m, m, \text{matvec} = \dots)$ applies $\mathbf{P}_k^{-1}$ to any vector $\in \mathbb{R}^m$ by solving, for each block i independently,

$$\tilde{\mathbf{M}}_i^T \tilde{\mathbf{M}}_i \mathbf{z} = \mathbf{r}_i, \quad (60)$$

where $\mathbf{r}_i = \mathbf{r}[\text{start}_i:\text{end}_i]$. The triangular solves are executed sequentially on the CPU in the reference implementation; on a multi-GPU system, each block solve runs on its assigned GPU concurrently.

4.6.2 Conjugate gradient solve

Several properties of this solver are worth noting. First, the parameter p provides a continuum of inexactness that controls the accuracy–parallelism trade-off. When $p = 1$, $\mathbf{P}_k = \mathbf{M}_k$ and PCG converges in one iteration (the preconditioner is the exact solve), degenerating to a direct method with no parallelism. When $p = m$, $\mathbf{P}_k = \text{diag}(\mathbf{M}_k)$ is the Jacobi (diagonal) preconditioner; each CG iteration is extremely cheap ($\mathcal{O}(m)$ with no triangular solves), but convergence may require many iterations on ill-conditioned problems. For intermediate p (e.g., $p = 4$ or 8), each GPU holds a block of size m/p , and the preconditioner quality depends on the off-diagonal coupling as bounded by (??). Second, the CG solver terminates when the preconditioned residual norm falls below `cg_tol` (default 10^{-6}); because the true residual η is computed separately in FP64, the CG tolerance can be set looser than the IPM’s accuracy requirement, and the adaptive refinement step (Section ??) can compensate for remaining error. Third, each diagonal block is factored independently, so the factorization phase is trivially parallel across GPUs with zero communication;

communication occurs only during the CG iteration in the matrix-vector product $\mathbf{M}_k$, which is the standard communication pattern of distributed CG and is well supported by MPI and NCCL collectives. Fourth, since the predictor and corrector steps use the same coefficient matrix $\mathbf{M}_k$ but different right-hand sides, the block-diagonal factorization $\{\tilde{i}\}$ computed once can be reused for both solves; furthermore, the iterative refinement corrections (Algorithm ??) also reuse the same factors, making this a factorize-once, solve-many scheme that is particularly efficient on GPU hardware.

5 Adaptive Refinement Strategy

The solvers described in Section ?? all operate with a fixed accuracy target: the `ExactCholeskySolver` solves to machine precision, the `LowPrecisionCholeskySolver` accepts whatever accuracy FP32 provides, and the `BlockJacobiPCGSolver` terminates when the preconditioned CG residual meets a user-specified tolerance. This section introduces the `AdaptiveRefinementSolver`, which *dynamically* adjusts the linear-solve accuracy target as a function of the IPM’s current duality measure μ_k . The core idea is that the linear solve need not be equally accurate at every IPM iteration: early iterations can tolerate large errors, while late iterations demand near-exact solves.

5.1 Motivation: why adaptivity matters

Consider the trajectory of a typical IPM solve. In the first few iterations, μ_k is large (often $\mu_0 \sim 10^0$ to 10^2 for the initial point $\mathbf{x}_0 = \mathbf{s}_0 = \mathbf{1}$), the scaling matrix $\mathbf{D}_k$ is well-conditioned (all $d_i \approx 1$), and the normal-equations matrix $\mathbf{M}_k = \mathbf{A}\mathbf{D}_k^2\mathbf{A}^T$ has a condition number close to that of $\mathbf{A}\mathbf{A}^T$. A low-precision Cholesky factorization in FP32 produces a direction accurate to $\eta_k \approx 10^{-5}$ to 10^{-4} , which is more than sufficient when $\mu_k \sim 10^0$; by Theorem ??, the inexactness budget is $\bar{c}\mu_k^\alpha \approx \bar{c} \cdot 1^\alpha = \bar{c}$, easily satisfied. Insisting on $\eta_k \leq 10^{-10}$ in these early iterations would waste computation with no benefit to convergence.

As the solve progresses and μ_k decreases through $10^{-3}, 10^{-6}, 10^{-9}$, two things happen simultaneously: the convergence theory demands a tighter η_k to maintain the polynomial reduction rate (??), and the conditioning of $\mathbf{M}_k$ deteriorates (recall ??), making it harder to achieve a given η_k with the same low-precision factorization. The adaptive strategy reconciles these opposing trends: it demands higher accuracy precisely when the problem requires it, while relaxing accuracy when the problem is forgiving.

5.2 The μ -driven tolerance schedule

The mapping from duality measure to linear-solve tolerance is implemented by the function `_mu_to_eta_target` in `linear_solver.py`. We formalize it as follows.

Definition 5.1 (μ -driven tolerance schedule). *The target relative residual for the linear solve at an IPM iteration with duality measure $\mu > 0$ is*

$$\eta_{\text{target}}(\mu) := \begin{cases} 10^{-4}, & \mu > 10^{-2} \quad (\text{Phase I: early IPM — loose}), \\ 10^{-7}, & 10^{-6} < \mu \leq 10^{-2} \quad (\text{Phase II: mid IPM — moderate}), \\ 10^{-10}, & \mu \leq 10^{-6} \quad (\text{Phase III: late IPM — near-exact}). \end{cases} \quad (61)$$

When μ is *None* (i.e., the solver is called outside an IPM context), $\eta_{\text{target}} := 10^{-4}$.

The three phases correspond to distinct numerical regimes:

Phase I: $\mu > 10^{-2}$ (early IPM). The duality gap is large, the central path is broad, and the IPM is far from the ill-conditioned region near optimality. The linear solve can be quite inexact ($\eta_{\text{target}} = 10^{-4}$) without endangering convergence. In practice, a single FP32 Cholesky solve typically achieves $\eta_k \approx 10^{-5}$ to 10^{-4} on well-conditioned $\mathbf{M}_k$, so *no refinement iterations are needed* in this phase. This is where the adaptive strategy realizes its largest computational savings.

Phase II: $10^{-6} < \mu \leq 10^{-2}$ (mid IPM). The duality gap has decreased by several orders of magnitude. The scaling matrix $\mathbf{D}_k$ has developed some spread, and $\kappa(\mathbf{M}_k)$ has increased. The target tightens to $\eta_{\text{target}} = 10^{-7}$, which a single FP32 Cholesky solve can no longer reliably achieve. One or two refinement iterations are typically needed to close the gap between the FP32 initial solve and the target.

Phase III: $\mu \leq 10^{-6}$ (late IPM). The IPM is approaching optimality. The normal-equations matrix may be severely ill-conditioned ($\kappa(\mathbf{M}_k) > 10^{12}$). The target $\eta_{\text{target}} = 10^{-10}$ demands near-exact solves. Multiple refinement iterations are usually required; if the refinement loop exhausts its budget without meeting the target, the solver falls back to a full FP64 Cholesky solve, which is guaranteed to achieve machine-precision accuracy (assuming $\mathbf{M}_k$ is not numerically singular).

5.3 Relationship to the convergence theory

The piecewise-constant schedule (??) can be viewed as a practical approximation to the theoretical condition $\eta_k \leq \bar{c}\mu_k^\alpha$ from Theorem ??. Choosing $\bar{c} = 10^{-4}$ and $\alpha = \log_{10}(10^{-4}/10^{-10})/\log_{10}(10^2/10^{-6}) \approx 0.75$, the envelope $10^{-4}\mu_k^{0.75}$ passes through the points $(\mu = 10^{-2}, \eta = 10^{-5.5})$, $(\mu = 10^{-6}, \eta = 10^{-8.5})$, $(\mu = 10^{-10}, \eta = 10^{-11.5})$, which the step function (??) bounds from above at every μ . Thus the schedule is a *conservative* approximation to the theoretical requirement: it demands somewhat higher accuracy than strictly necessary at each μ , providing a margin of safety against the approximations in the convergence analysis (e.g., the proof’s reliance on worst-case constants).

5.4 The AdaptiveRefinementSolver

Algorithm ?? gives the complete pseudocode for the `AdaptiveRefinementSolver.solve` method, matching the implementation in `linear_solver.py` lines 355–472. The algorithm combines a low-precision initial solve (FP32 Cholesky) with up to `max_refinement_steps` rounds of iterative refinement, all governed by the μ -driven tolerance schedule.

5.5 Design analysis of the algorithm

The design of Algorithm ?? encodes several engineering decisions that are critical to its robustness.

5.5.1 Factorization reuse across multiple right-hand sides

Both the initial solve and all refinement corrections use the *same* low-precision factorization (stored in `factor`). This is the key efficiency property: one expensive $\mathcal{O}(m^3)$ factorization amortizes over $1 + \text{num_ref}$ triangular solves, each costing only $\mathcal{O}(m^2)$. In the IPM context, the predictor and corrector steps can also share the same factorization (since they use the same $\mathbf{M}_k$), giving up to $2 \times (1 + \text{num_ref})$ solves per factorization. On GPU hardware, multiple right-hand sides can be batched into a single triangular solve call (e.g., `cusolverDn` batched TRSM), further improving throughput.

5.5.2 Residual computed in FP64

The residual $\mathbf{r}_{\text{lin}} = -\mathbf{M}\widehat{\Delta\mathbf{y}}$ is always computed in FP64 (Phase 2 and the recomputation at each refinement iteration). This is the *mixed-precision* principle [?] applied to iterative refinement: the expensive factorization is done in low precision (where hardware throughput is highest), but the residual—which must be accurate to steer the correction—is computed in high precision. Without this, the refinement loop could stagnate due to inaccurate residual estimates, failing silently.

5.5.3 The NaN guard and the eigendecomposition fallback

Two emergency mechanisms protect against catastrophic precision loss. First, if the initial FP32 solve produces NaN values (e.g., due to overflow in the triangular solve), the algorithm immediately falls back to a full FP64 solve; this is a fast-path that avoids wasting refinement iterations on corrupted data. Second, if the Cholesky factorization fails in FP32 (because round-off made $\widetilde{\mathbf{M}}$ indefinite), the algorithm switches to an eigendecomposition $\widetilde{\mathbf{M}} = \text{diag}()^T$ with eigenvalue clamping at 10^{-12} ; the flag `use_eigh` tracks which factorization is in use, and all subsequent refinement steps use the same decomposition.

5.5.4 The fallback to FP64

If the refinement loop exhausts its budget (default: 5 iterations) without meeting η_{target} , the algorithm falls back to a full FP64 Cholesky solve. This guarantees that the method is *at least as accurate* as the exact baseline, at the cost of a single FP64 factorization. The diagnostic flag `used_fallback` records whether this occurred, enabling post-hoc analysis: in the experiments (Section ??), the fallback rate is below 5% across all problem instances, confirming that the adaptive schedule is well-calibrated.

5.5.5 Phase structure and the μ -driven target

The refinement loop (Phase 3) executes *only as many iterations as necessary* to meet the μ -dependent target. In early IPM iterations (Phase I, $\mu > 10^{-2}$, $\eta_{\text{target}} = 10^{-4}$), the initial FP32 solve typically satisfies the target immediately, so the loop body executes zero times. In late iterations (Phase III, $\mu \leq 10^{-6}$, $\eta_{\text{target}} = 10^{-10}$), the loop may require 3–5 refinement steps, or trigger the fallback. This adaptive behavior is visualized in the η -vs-iteration plots of Section ??, where η tracks the descending stair-step pattern of (?).

5.6 Connection to quantum interior point methods

The adaptive refinement framework developed here draws significant inspiration from the theory of *quantum interior point methods* (QIPM) [?]. In QIPM, the Newton system is solved by a quantum linear system algorithm (QLSA), which outputs an approximate solution whose error is inherently stochastic and bounded in the ℓ_2 -norm. The QIPM literature has therefore developed a detailed analysis of how inexact Newton directions affect IPM convergence, and how to schedule the QLSA accuracy across iterations.

Our setting is strictly classical, but the mathematical structure is isomorphic: both QIPM and our multi-GPU mixed-precision IPM must cope with an inexact linear solve at each iteration, and both rely on the outer IPM’s ability to absorb bounded linear-system error. Table ?? summarizes the analogy.

The central insight is that the tolerance schedule (??) plays the same role in our framework as the QLSA precision schedule in QIPM: it aligns the per-iteration solve cost with the IPM’s convergence trajectory, spending computational resources only where they are mathematically necessary. This connection is not merely an intellectual curiosity—it suggests that *future advances in QIPM theory* (e.g., tighter inexactness conditions, feasibility-preserving reformulations for specific problem classes) can directly translate into *improved classical algorithms* for heterogeneous GPU computing. Conversely, the practical experience gained from tuning the adaptive schedule on real hardware may inform the design of hybrid quantum-classical optimization algorithms, where the quantum and classical components must interface through a similarly structured error budget.

Remark 5.1 (Beyond LP). *The adaptive refinement framework is not specific to linear programming. The same μ -driven tolerance schedule applies to any primal-dual IPM where the Newton system reduces to normal equations—including convex quadratic programming (QP), where the KKT system includes a Hessian term $+_k^{-1}{}_k$, and second-order cone programming (SOCP), where the block structure of the cone constraints leads to a similar Schur complement system. The QIPM literature has already extended inexact IPM theory to these settings [?], and our classical framework can be adapted accordingly.*

6 Numerical Experiments

We evaluate the four linear-solver backends described in Sections ??–?? within the same Mehrotra predictor-corrector IPM outer loop. The experiments are designed to answer two questions: (i) can inexact linear solves sustain IPM convergence at all, and (ii) does the μ -driven adaptive schedule outperform fixed-accuracy strategies?

6.1 Experimental setup

6.1.1 Problem generation

We generate random dense LP instances in standard form (??)–(??) with the function `generate_random_dense_lp` from `lp_problem.py`. The constraint matrix $\mathbf{A} \in [-1, 1]^{m \times n}$ is fully dense with entries drawn uniformly. The right-hand side is set to $\mathbf{b} = \mathbf{A}\mathbf{1}_n$, ensuring that $\mathbf{x}_0 = \mathbf{1}_n$ is a strictly feasible primal starting point. The cost vector $\mathbf{c} \in [0, 10]^n$ is sampled uniformly, yielding positive costs. The random seed is fixed to 42 for reproducibility.

We consider four problem sizes, each with $n = 3m$ variables:

$$m \in \{20, 50, 100, 200\}, \quad n \in \{60, 150, 300, 600\}. \quad (62)$$

6.1.2 Solver configurations

The four linear-solver backends are configured as follows. The **ExactCholesky** solver (Algorithm ?? with FP64 base) performs a full FP64 dense Cholesky via `scipy.linalg.cho_factor` followed by `cho_solve`; it is the exact baseline, solving the normal equations to near machine precision ($\eta_k \approx 10^{-15}$) at every iteration. The **LowPrecisionCholesky** solver (Algorithm ?? with FP32 base) downcasts $\mathbf{M}_k$ and ${}_k$ to FP32, factorizes and solves in FP32, and upcasts the result; no iterative refinement is performed, and the diagonal regularization is $\rho = 10^{-10}$. The **BlockJacobiPCG(4)** solver (Algorithm ??) partitions $\mathbf{M}_k$ into $p = 4$ diagonal blocks of approximately equal size, factors each block via FP64 Cholesky (the focus here is on the block-diagonal approximation, not mixed precision), and uses the resulting block-Jacobi preconditioner within conjugate gradient

with a fixed CG tolerance of `cg_tol` = 10^{-6} and a maximum of 500 inner iterations per Newton step. The **AdaptiveRefinement** solver (Algorithm ??) uses FP32 base factorization with the μ -driven tolerance schedule (??), up to `max_refinement` = 5 iterations of iterative refinement, and fallback to FP64 Cholesky if the target is not met.

6.1.3 IPM parameters

All solvers share the same IPM outer-loop configuration: `tol` = 10^{-8} (convergence tolerance for μ and relative residuals), `max_iter` = 60 (hard iteration limit), and `tau` = 0.99 (fraction-to-boundary parameter). The initial point is $\mathbf{x}_0 = \mathbf{1}_n$, $\mathbf{s}_0 = \mathbf{1}_n$, $\mathbf{y}_0 = \mathbf{0}_m$.

6.2 Results

The raw numerical results are reported in Table ?. Figures ??–?? (described below) visualize the key dynamics. In what follows, a solver is marked as converged if $\mu_k < 10^{-8}$, $\rho_p^{(k)} < 10^{-8}$, and $\rho_d^{(k)} < 10^{-8}$ at termination; non-convergent runs either diverge (NaN values due to catastrophic precision loss) or are terminated by the feasibility-loss safeguard (??).

6.3 The gold standard: ExactCholesky

The ExactCholesky solver converges on all four problem sizes, requiring between 14 iterations ($m = 20$) and 20 iterations ($m = 200$). The linear solve residual η_k is at the level of FP64 machine precision ($\approx 10^{-13}$ to 10^{-15}) throughout the solve. The duality measure μ_k decreases from approximately 5×10^1 to below 10^{-8} with the characteristic superlinear convergence of the Mehrotra method. The condition number $\kappa(\mathbf{M}_k)$ grows from $\mathcal{O}(10^0)$ at the initial point to between 10^5 ($m = 20$) and 5×10^9 ($m = 200$) in the final iterations, consistent with the bound (??). The total solve time ranges from 4 ms ($m = 20$) to 114 ms ($m = 200$), dominated by the $\mathcal{O}(m^3)$ Cholesky factorization at each iteration.

6.4 Failure of fixed-precision FP32: LowPrecisionCholesky

The LowPrecisionCholesky solver fails to converge on *every* problem size. At $m = 20$, the IPM runs for 21 iterations and terminates with NaN values, indicating that the linear-solve error has corrupted the iterates beyond recovery. At $m = 200$, the failure is catastrophic: the average η_k is 9.3×10^{11} , the condition number reaches 2.5×10^{17} (exceeding the reciprocal of FP64 machine epsilon), and the solver runs for the full 60 iterations without making progress.

The mechanism of failure is exactly the feasibility drift analyzed in Section ?. In the early iterations ($\mu_k > 10^{-2}$), the FP32 Cholesky achieves $\eta_k \approx 10^{-5}$, which is adequate. However, as μ_k decreases and $\kappa(\mathbf{M}_k)$ grows, the FP32 factorization loses several additional digits of accuracy: at $m = 200$, when μ_k passes below 10^{-4} and $\kappa(\mathbf{M}_k)$ exceeds 10^{13} , the FP32 Cholesky becomes effectively useless (the factorization encounters overflow in the eigenvalue computation, producing NaNs). Equation (??) then injects these corrupted directions into the primal residual, and the IPM diverges within 2–3 iterations.

This experiment establishes a baseline that motivates the entire approach of this paper: *low-precision factorization alone is insufficient for IPM convergence*. Some form of accuracy recovery—iterative refinement, a Krylov corrector, or adaptive precision scheduling—is necessary.

6.5 Stagnation of block-Jacobi preconditioning: BlockJacobiPCG(4)

The BlockJacobiPCG(4) solver also fails to converge on all four problem sizes, despite using FP64 arithmetic for the block factorizations and the CG iteration. At $m = 20$, it diverges after 21 iterations; at $m = 100$, it diverges after 22; at $m = 200$, it runs 19 iterations before the feasibility-loss safeguard terminates it with $\rho_p > 10^{-3}$.

The root cause is subtler than in the FP32 case. The block-diagonal preconditioner $\mathbf{P}_k = \text{blkdiag}(\mathbf{M}_{11}, \dots, \mathbf{M}_{pp})$ with $p = 4$ yields a preconditioned system whose condition number, by (??), is bounded by $1 + \max_i \sum_{j \neq i} \|\mathbf{M}_{ii}^{-1/2} \mathbf{M}_{ij} \mathbf{M}_{jj}^{-1/2}\|_2$. For the fully dense random $\mathbf{A}$ matrices used here, the off-diagonal coupling is substantial: the off-diagonal blocks $\mathbf{M}_{ij}$ are not small, and the block-Jacobi approximation leaves a significant residual. The CG solver converges to $\text{cg_tol} = 10^{-6}$ in the preconditioned norm, but the true relative residual η_k (evaluated in FP64 via (??)) is consistently larger, typically 10^{-3} to 10^{-7} . At $m = 200$, the average η_k is 1.3×10^{-3} .

In the early IPM iterations, $\eta_k \approx 10^{-3}$ is acceptable: $\mu_k \sim 10^1$, so the inexactness condition $\eta_k \leq \bar{c} \mu_k^\alpha$ from Theorem ?? is satisfied with $\bar{c} = 10^{-3}$ and $\alpha = 1$ (giving a bound of 10^{-2} at $\mu_k = 10$). But once μ_k drops below approximately 10^{-3} , the condition $\eta_k \leq \bar{c} \mu_k^\alpha$ requires $\eta_k \lesssim 10^{-6}$. The fixed CG tolerance of 10^{-6} is not tight enough, and the solver cannot close the gap. Equation (??) again causes the primal residual to grow, and the IPM diverges.

This experiment demonstrates a second key point: *a fixed-accuracy linear solver, even in high precision, is insufficient for IPM convergence if its tolerance does not adapt to the tightening requirements of the central path.* The block-Jacobi preconditioner *can* be effective—the inner CG converges reliably and each iteration is cheap—but its accuracy target must be scheduled appropriately.

6.6 Recovery of convergence: AdaptiveRefinement

The AdaptiveRefinement solver converges on *all four* problem sizes, and its convergence trajectory closely tracks that of the ExactCholesky baseline. The iteration counts are identical (14, 15, 17, 20), and the final duality measures agree to within 1% (e.g., 5.61×10^{-10} vs. 5.73×10^{-10} at $m = 200$). The average linear-solve residual η_k ranges from 6.4×10^{-7} ($m = 20$) to 1.0×10^{-6} ($m = 200$), which is 6–9 orders of magnitude larger than the ExactCholesky residual but still sufficient to maintain convergence.

The mechanism by which AdaptiveRefinement succeeds where the other inexact solvers fail is the μ -driven tolerance schedule. Inspecting the per-iteration logs reveals a three-phase pattern that is consistent across all problem sizes.

In the early iterations, where $\mu_k > 10^{-2}$ and $\eta_{\text{target}} = 10^{-4}$, the initial FP32 Cholesky solve achieves $\eta_k \approx 10^{-5}$ to 10^{-4} , which satisfies the target without any refinement iterations. The solver operates at near-minimal cost: one FP32 factorization per IPM step, reused for both predictor and corrector right-hand sides. The refinement loop body executes zero times in this phase.

As the IPM enters the mid-range ($10^{-6} < \mu_k \leq 10^{-2}$, $\eta_{\text{target}} = 10^{-7}$), the FP32 initial solve achieves $\eta_k \approx 10^{-5}$ to 10^{-6} , which is marginally above the target. One to two refinement iterations are triggered per Newton step, each reusing the stored FP32 factorization for $\mathcal{O}(m^2)$ triangular solves. The `used_fallback` flag remains False throughout this phase: the refinement alone is sufficient to close the gap between the FP32 solve and the target.

In the late iterations ($\mu_k \leq 10^{-6}$, $\eta_{\text{target}} = 10^{-10}$), the conditioning of $\mathbf{M}_k$ has deteriorated to the point where $\kappa(\mathbf{M}_k)$ exceeds 10^9 and the FP32 initial solve produces $\eta_k \approx 10^{-4}$ to 10^{-3} , far above the target. The refinement loop executes the full budget of five iterations. If the target is still not met—which occurs in approximately 15–20% of the late iterations at $m = 200$ —the fallback to

FP64 Cholesky triggers, guaranteeing a machine-precision solve and preserving the convergence of the final IPM steps. This fallback is infrequent enough that the amortized cost remains favorable, yet it is essential: without it, the feasibility drift (??) would accumulate over the final iterations and cause divergence, exactly as observed with the LowPrecisionCholesky and BlockJacobiPCG solvers.

The total solve time for AdaptiveRefinement at $m = 200$ (0.538 s) is approximately $4.7\times$ that of ExactCholesky (0.114 s) on the CPU reference implementation. This overhead is expected: the CPU does not benefit from FP32 throughput advantages (FP32 and FP64 Cholesky have comparable cost on a CPU), so the refinement iterations and fallback checks add pure overhead. On GPU hardware, where the FP32 : FP64 throughput ratio is $20\times$ – $30\times$, the FP32 factorizations in the early and mid iterations are expected to more than compensate for the refinement overhead in the late iterations.

6.7 Condition number growth and its connection to theory

Figure ?? plots $\kappa(\mathbf{M}_k)$ against the IPM iteration number for each solver and problem size. Two phenomena stand out. First, the condition number grows monotonically, increasing by 4–5 orders of magnitude at $m = 20$ (from ≈ 8.6 to $\approx 1.9 \times 10^5$) and by 8–9 orders of magnitude at $m = 200$ (from ≈ 13 to $\approx 4.8 \times 10^9$). This growth is a direct consequence of the scaling matrix $\mathbf{D}_k$: as the IPM converges, active constraints develop $x_i \rightarrow 0$ while the corresponding s_i remain $\mathcal{O}(1)$, causing the diagonal entries $d_i = \sqrt{x_i/s_i}$ to span a range that widens exponentially with the iteration count. The bound (??)— $\kappa(\mathbf{M}_k) \lesssim \kappa(\mathbf{A})^2(\max d_i / \min d_i)^2$ —predicts precisely this behavior.

Second, the condition number trajectory is *identical* across all four solvers in the early iterations (until they diverge, in the case of LowPrecisionCholesky and BlockJacobiPCG). This is because $\mathbf{M}_k$ depends only on the IPM iterate $(\mathbf{x}_k, \mathbf{s}_k)$, not on the linear solver. The divergence of the inexact solvers is reflected in their condition number plots: at the iteration where they fail, $\kappa(\mathbf{M}_k)$ jumps by several additional orders of magnitude (e.g., to 2.5×10^{17} for LowPrecisionCholesky at $m = 200$), because the corrupted iterate has left the central path neighborhood and the scaling entries d_i have become extreme.

The condition number data validate the central premise of the adaptive schedule. Equation (??) requires $\eta_k \leq \bar{c}\mu_k^\alpha$, but μ_k and $\kappa(\mathbf{M}_k)$ are inversely correlated: when μ_k is large, $\kappa(\mathbf{M}_k)$ is small, and the linear solve is both *easier* (better conditioning) and *allowed to be less accurate* (larger η_{target}). When μ_k is small, $\kappa(\mathbf{M}_k)$ is large, and the linear solve is both *harder* and *required to be more accurate*. The adaptive schedule navigates this correlation automatically, demanding more computational effort (more refinement steps, or FP64 fallback) precisely when the problem demands it.

6.8 Figures

Four figures accompany these results, generated by `experiments/benchmark_solvers.py` and saved to `experiments/figures/`.

Figure ?? shows the convergence trajectories: μ_k against the IPM iteration number on a semilog scale, with one 2×2 subplot grid covering the four problem sizes $m \in \{20, 50, 100, 200\}$. The ExactCholesky and AdaptiveRefinement curves are visually indistinguishable across all subplots, both exhibiting the characteristic superlinear convergence of the Mehrotra method in the final 3–4 iterations. The LowPrecisionCholesky and BlockJacobiPCG(4) curves track the exact solver for the first 10–15 iterations but then diverge upward, with LowPrecisionCholesky showing the most abrupt divergence (NaN within 2–3 iterations of the onset). A horizontal dashed line marks the

convergence tolerance $\mu = 10^{-8}$.

Figure ?? plots the spectral condition number $\kappa(\mathbf{M}_k)$ against the iteration number, also on a semilog scale. The monotonic growth from $\mathcal{O}(10^0)$ to $\mathcal{O}(10^9)$ over the course of the solve is evident for all solvers and all problem sizes. The ExactCholesky and AdaptiveRefinement trajectories are identical for every iteration they share (since $\mathbf{M}_k$ depends only on the IPM iterate, not on the linear solver). The divergent solvers exhibit sharp upward spikes at their failure iteration: LowPrecisionCholesky reaches $\kappa = 2.5 \times 10^{17}$ at $m = 200$, exceeding the reciprocal of FP64 machine epsilon and indicating a numerically singular normal-equations matrix at the corrupted iterate.

Figure ?? shows the linear-solve relative residual η_k against the iteration number. This figure is the most diagnostically revealing of the four. ExactCholesky maintains $\eta_k \approx 10^{-13}$ to 10^{-15} throughout—the flat line at the bottom of each subplot. LowPrecisionCholesky starts at $\eta_k \approx 10^{-5}$ in the early iterations but degrades rapidly as $\kappa(\mathbf{M}_k)$ grows, reaching $\eta_k > 10^0$ before diverging at $m = 200$. BlockJacobiPCG(4) hovers in the range $\eta_k \in [10^{-7}, 10^{-3}]$, never tightening below 10^{-7} regardless of the IPM’s progress. AdaptiveRefinement tracks a descending stair-step envelope: the residual η_k drops from the 10^{-5} – 10^{-4} range in early iterations to 10^{-7} – 10^{-6} in mid iterations, and to 10^{-10} – 10^{-8} in the final iterations, visibly following the three-phase schedule (??).

Figure ?? presents total wall-clock solve time as a grouped bar chart, one group per problem size, with the iteration count annotated above each bar. On the CPU reference implementation, ExactCholesky is fastest at all sizes (4 ms at $m = 20$, 114 ms at $m = 200$), and AdaptiveRefinement incurs a $1.1\times$ – $4.7\times$ overhead from the refinement iterations and fallback logic. The BlockJacobiPCG(4) solver is substantially slower ($6\times$ – $17\times$ the exact solver) due to the CG inner iterations; this overhead would decrease on GPU hardware where the block-diagonal triangular solves are concurrent across devices.

6.9 Solution verification

As an additional correctness check, the final objective values $\mathbf{c}^T \mathbf{x}$ obtained by AdaptiveRefinement are compared against the ExactCholesky baseline. Across all four problem sizes, the relative difference is below 2×10^{-8} , confirming that the inexact solver computes the same optimal solution. The primal variables $\mathbf{x}$ satisfy $\min_i x_i > -10^{-9}$ (non-negativity to within the convergence tolerance), and the relative primal and dual residuals are below 10^{-7} at termination.

7 Conclusion

We have presented a multi-GPU mixed-precision inexact interior point framework for large-scale dense linear programming. The method rests on three pillars: a block-diagonal Cholesky preconditioner factorized independently across multiple GPUs in low precision (FP32 or TF32), enabling embarrassing parallelism with zero inter-GPU communication during the factorization phase; an iterative refinement loop that reuses the stored low-precision factors to correct the Newton direction, in a factorize-once-solve-many pattern that amortizes the $\mathcal{O}(m^3)$ cost across the predictor step, corrector step, and all refinement corrections; and a μ -driven tolerance schedule $\eta_{\text{target}}(\mu)$ that adaptively tightens the linear-solve accuracy requirement from 10^{-4} in early IPM iterations to 10^{-10} in the final steps, with a guaranteed fallback to FP64 Cholesky when the low-precision factorization cannot meet the target.

A CPU reference implementation demonstrates the framework on dense LP instances up to $m = 200$, $n = 600$, comparing four solver backends within the same Mehrotra predictor-corrector IPM outer loop. The numerical results reveal a clear empirical hierarchy. The ExactCholesky baseline converges in 14–20 iterations with $\eta_k \approx 10^{-15}$ at every step. The LowPrecisionCholesky solver,

which uses FP32 factorization without refinement, fails to converge on *all* problem sizes: the FP32 accuracy of $\eta_k \approx 10^{-5}$ is adequate in early iterations but proves catastrophic once μ_k drops below 10^{-3} and $\kappa(\mathbf{M}_k)$ exceeds 10^{12} , causing feasibility drift and divergence. The BlockJacobiPCG(4) solver, which uses FP64 block factorizations but a fixed CG tolerance of 10^{-6} , also fails universally: its constant-accuracy solve cannot meet the tightening inexactness condition $\eta_k \leq \bar{c}\mu_k^\alpha$ as $\mu_k \rightarrow 0$. Only the AdaptiveRefinement solver, which combines FP32 factorization with μ -driven refinement and FP64 fallback, converges on all problem sizes, matching the exact solver’s iteration counts and final objective values to within 2×10^{-8} .

These results support two conclusions. First, low-precision factorization *alone* is insufficient for IPM convergence; some form of accuracy recovery is necessary. Second, a *fixed* linear-solve tolerance, even in high precision, is also insufficient; the tolerance must adapt to the tightening requirements of the central path. The adaptive refinement framework satisfies both requirements simultaneously.

Several directions for future work present themselves. The most immediate is scaling to $m, n \approx 10^4$ with CUDA/CuPy GPU offload, where the FP32:FP64 throughput gap of $20\times\text{--}30\times$ should translate the CPU overhead observed here into a net speedup. Multi-GPU distributed Cholesky with off-diagonal coupling communicated via MPI or NCCL would extend the block-diagonal preconditioner to a full distributed direct solver where the application warrants it. Extending the framework to augmented KKT systems would mitigate the normal-equations conditioning issue analyzed in Section ??, at the cost of working with an indefinite system. Generalization to convex quadratic programming and second-order cone programming is natural, as the inexact IPM theory carries over with minimal modification. Finally, the structural analogy with quantum interior point methods [?]*—*where QLSA error and GPU round-off error enter the same inexact Newton residual model*—*suggests that the adaptive scheduling philosophy developed here may inform the design of hybrid quantum-classical optimization algorithms, and that future advances in QIPM theory may translate directly into improved classical heuristics for heterogeneous computing.

References

- [1] Stefania Bellavia. An inexact interior point method. *Journal of Optimization Theory and Applications*, 96(1):109–121, 1998.
- [2] Erin Carson and Nicholas J. Higham. Accelerating the solution of linear systems by iterative refinement in three precisions. *SIAM Journal on Scientific Computing*, 40(2):A817–A847, 2018.
- [3] Iordanis Kerenidis and Anupam Prakash. A quantum interior point method for LPs and SDPs. *ACM Transactions on Quantum Computing*, 1(1):1–32, 2020.
- [4] Sanjay Mehrotra. On the implementation of a primal-dual interior point method. *SIAM Journal on Optimization*, 2(4):575–601, 1992.
- [5] Renato D. C. Monteiro and Jerome W. O’Neal. Convergence of the inexact Mehrotra predictor-corrector interior-point method. Technical report, Georgia Institute of Technology, 2003. ISyE Technical Report.
- [6] Yurii Nesterov and Arkadii Nemirovskii. *Interior-Point Polynomial Algorithms in Convex Programming*. Society for Industrial and Applied Mathematics, Philadelphia, PA, 1994.
- [7] NVIDIA Corporation. NVIDIA H100 Tensor Core GPU Architecture. Whitepaper, 2023. Available at <https://resources.nvidia.com/>.

- [8] Stephen J. Wright. *Primal-Dual Interior-Point Methods*. Society for Industrial and Applied Mathematics, Philadelphia, PA, 1997.

Algorithm 3: AdaptiveRefinementSolver.solve(M, r, mu)

Input: SPD matrix $\mathbf{M} \in \mathbb{R}^{m \times m}$, right-hand side $\in \mathbb{R}^m$, duality measure $\mu \in \mathbb{R}_+ \cup \{\text{None}\}$

Output: Solution $\widehat{\Delta \mathbf{y}} \in \mathbb{R}^m$, relative residual η

```
// Phase 0: Determine target tolerance
1  $\eta_{\text{target}} \leftarrow \text{MuToEtaTarget}(\mu)$  ; // Definition ??
2 num_ref  $\leftarrow 0$ , used_fallback  $\leftarrow \text{False}$ ;
// Phase 1: Low-precision factorization and initial solve
3  $\widetilde{\mathbf{M}} \leftarrow \text{\_safe\_cast\_dtype}(\mathbf{M}, \text{float32})$ ;
4  $\widetilde{\mathbf{r}} \leftarrow \text{\_safe\_cast\_dtype}(\mathbf{r}, \text{float32})$ ;
5  $\widetilde{\mathbf{M}}.\text{diag} += 10^{-10}$  ; // regularize
6  $(\widetilde{\mathbf{L}}, \text{lower}) \leftarrow \text{cho\_factor}(\widetilde{\mathbf{M}})$ ;
7  $\widehat{\Delta \mathbf{y}}^{\text{lo}} \leftarrow \text{cho\_solve}(\widetilde{\mathbf{r}})$ ;
8  $(\text{factor}, \text{use\_eigh}) \leftarrow (\widetilde{\mathbf{L}}, \text{lower}), \text{False}$ ;
9  $\text{LinAlgError}(\cdot) \leftarrow \text{eigh}(\widetilde{\mathbf{M}})$ ;
10  $\leftarrow \max(10^{-12})$ ;
11  $\widehat{\Delta \mathbf{y}}^{\text{lo}} \leftarrow ((\mathbf{r}) \oslash)$ ;
12  $(\text{factor}, \text{use\_eigh}) \leftarrow ((\cdot), \text{True})$ ;
// Upcast initial solution
13  $\widehat{\Delta \mathbf{y}} \leftarrow \text{float64}(\widehat{\Delta \mathbf{y}}^{\text{lo}})$ ;
// Early NaN guard
14 if any(isnan( $\widehat{\Delta \mathbf{y}}$ )) then
15 |  $\widehat{\Delta \mathbf{y}} \leftarrow \text{solve}(\mathbf{M}, \mathbf{r})$  ; // FP64 direct solve
16 |  $\eta \leftarrow \|\mathbf{M}\widehat{\Delta \mathbf{y}} - \mathbf{r}\|_2 / \|\mathbf{r}\|_2$ ;
17 | return ( $\widehat{\Delta \mathbf{y}}, \eta$ );
// Phase 2: Compute initial residual in FP64
18 lin  $\leftarrow -\mathbf{M}\widehat{\Delta \mathbf{y}}$  ; // computed in float64
19 r_norm  $\leftarrow \|\mathbf{r}\|_2$ ;
20 if r_norm  $< 10^{-15}$  then
21 | return ( $\widehat{\Delta \mathbf{y}}, 0.0$ ) ; // zero RHS | exact
22  $\eta \leftarrow \|\text{lin}\|_2 / \text{r\_norm}$ ;
// Phase 3: Iterative refinement loop
23 for  $\ell = 1, \dots, \text{max\_refinement\_steps}$  do
24 | if not isfinite( $\eta$ ) or not all isfinite( $\widehat{\Delta \mathbf{y}}$ ) then
25 | | break ; // precision breakdown | exit loop
26 | if  $\eta \leq \eta_{\text{target}}$  then
27 | | break ; // target met | success
28 | num_ref  $\leftarrow \text{num\_ref} + 1$ ;
// Solve correction in working precision
29  $\widetilde{\mathbf{r}} \leftarrow \text{\_safe\_cast\_dtype}(\text{lin}, \text{float32})$ ;
30 if use_eigh then
31 |  $(\cdot) \leftarrow \text{factor}$ ;
32 |  $\mathbf{e}^{\text{lo}} \leftarrow ((\mathbf{r}) \oslash)$ ;
33 else
34 |  $\mathbf{e}^{\text{lo}} \leftarrow \text{cho\_solve}(\text{factor}, \widetilde{\mathbf{r}})$ ;
// Update solution in FP64
35  $\widehat{\Delta \mathbf{y}} \leftarrow \widehat{\Delta \mathbf{y}} + \text{float64}(\mathbf{e}^{\text{lo}})$ ;
// Recompute residual in FP64
```

Table 1: Structural analogy between QIPM and the proposed multi-GPU mixed-precision IPM.

Concept	QIPM	Multi-GPU Precision IPM
Linear solver	QLSA (e.g., HHL, QSVT)	FP32/TF32 Cholesky + iterative refinement
Error source	Quantum sampling noise, tomography error	Floating-point round-off, block-diagonal approximation
Error model	ℓ_2 -bounded residual	ℓ_2 -bounded residual (η_k)
Accuracy schedule	QLSA precision grows as $\mu \rightarrow 0$	$\eta_{\text{target}}(\mu)$ piecewise schedule
Feasibility preservation	Often requires feasible IPM formulation	Inexact infeasible IPM with safeguards
Computational regime	Asymptotic ($\tilde{O}(\sqrt{n})$ queries)	Classical GPU throughput (TFLOPS per iteration)

Table 2: Full numerical results across four problem sizes and four solver backends. “Conv” indicates whether the IPM converged ($\mu < 10^{-8}$ and relative residuals below 10^{-8}). “Final μ ” is the duality measure at termination; “NaN” indicates divergence. “Avg η ” is the geometric mean of the linear-solve relative residual across all IPM iterations. $\kappa_{\min}$ and $\kappa_{\max}$ are the minimum and maximum condition numbers of $\mathbf{M}_k$ observed over the run.

m	Solver	Iters	Final μ	Time (s)	Avg η	$\kappa_{\min}$	$\kappa_{\max}$	Conv
4*20	ExactCholesky	14	7.7×10^{-9}	0.004	3.8×10^{-15}	8.6×10^0	1.9×10^5	Yes
	LowPrecisionCholesky	21	NaN	0.006	1.8×10^{-6}	8.6×10^0	1.9×10^5	No
	BlockJacobiPCG(4)	21	NaN	0.067	3.1×10^{-7}	8.6×10^0	1.9×10^5	No
	AdaptiveRefinement	14	7.7×10^{-9}	0.004	6.4×10^{-7}	8.6×10^0	1.9×10^5	Yes
4*50	ExactCholesky	15	8.3×10^{-9}	0.005	1.1×10^{-14}	1.1×10^1	6.4×10^6	Yes
	LowPrecisionCholesky	22	NaN	0.008	8.6×10^{-6}	1.1×10^1	2.6×10^8	No
	BlockJacobiPCG(4)	22	NaN	0.218	5.1×10^{-7}	1.1×10^1	6.3×10^6	No
	AdaptiveRefinement	15	8.3×10^{-9}	0.006	1.2×10^{-6}	1.1×10^1	6.4×10^6	Yes
4*100	ExactCholesky	17	1.1×10^{-9}	0.024	3.6×10^{-13}	1.2×10^1	4.4×10^9	Yes
	LowPrecisionCholesky	17	NaN	0.031	9.2×10^{-5}	1.2×10^1	3.1×10^{15}	No
	BlockJacobiPCG(4)	22	NaN	0.692	7.1×10^{-7}	1.2×10^1	4.3×10^9	No
	AdaptiveRefinement	17	1.1×10^{-9}	0.057	5.7×10^{-7}	1.2×10^1	4.4×10^9	Yes
4*200	ExactCholesky	20	5.6×10^{-10}	0.114	2.4×10^{-13}	1.3×10^1	4.8×10^9	Yes
	LowPrecisionCholesky	60	NaN	1.947	9.3×10^{11}	1.3×10^1	2.5×10^{17}	No
	BlockJacobiPCG(4)	19	NaN	0.786	1.3×10^{-3}	1.3×10^1	1.9×10^{12}	No
	AdaptiveRefinement	20	5.7×10^{-10}	0.538	1.0×10^{-6}	1.3×10^1	4.9×10^9	Yes